
BOOTCAMP CIBERSEGURIDAD

Workshop

Módulo 3 – Centro de Operaciones
Submódulo: Operaciones SOC y Monitoreo

Implementación de una plataforma básica de SIEM y monitoreo de logs con Elastic Stack

Docente:	Giovanny Escobar Uribe
Estudiantes:	Cristian Camilo Perez Puentes
Fecha de entrega:	15 de junio de 2026
Palabras clave:	SIEM, Estaciones de trabajo, Logs, Reglas de seguridad, Monitoreo
Aplica a:	Desktop / Server

Contents

1	Descripción y objetivos	3
2	Fase 0: Preparativos y configuración de red	4
2.1	Problema de conectividad y solución	6
2.2	Verificación de conectividad entre las máquinas	6
3	Fase 1: Diseño del entorno	9
3.1	Resumen del diseño	9
4	Fase 2: Instalación de Elasticsearch y Kibana	10
4.1	Instalación de Elasticsearch en Kali Linux	10
4.2	Instalación de Kibana en Kali Linux	12
4.3	Verificación de que Elasticsearch recibe y responde consultas	15
5	Fase 3: Integración de eventos Windows con Winlogbeat	17
5.1	Instalación y configuración inicial	17
5.2	Problema de conexión a Kibana y solución	20
5.3	Inicio del servicio Winlogbeat	22
5.4	Activación de la auditoría en Windows	23
5.5	Confirmación en Kibana de que los eventos llegan	25
6	Fase 4: Integración de métricas Linux con Metricbeat	26
6.1	Configuración de la salida hacia Elasticsearch	26
6.2	Habilitación del módulo <code>system</code>	27
6.3	Validación de la configuración	27
6.4	Inicio del servicio	28
6.5	Verificación en Kibana de la llegada de métricas	28
7	Fase 5: Exploración de eventos en Kibana	29
7.1	Acceso a Discover	29
7.2	Filtrar eventos por host	29
7.3	Filtrar eventos por agente	29
7.4	Filtrar eventos por tipo de evento	30
7.5	Identificación de campos relevantes	31
7.6	Métricas de Metricbeat – filtrado por <code>event.dataset</code>	31
7.7	Comparación logs Windows vs métricas Linux	33
8	Fase 6: Generación controlada de eventos	36
8.1	Eventos controlados en Windows	36
8.1.1	Eventos del sistema – Process Termination (4689)	36
8.1.2	Eventos de seguridad – Firewall Rule Change (4948)	36
8.1.3	Evento de aplicación – custom (999)	37
8.1.4	Eventos de PowerShell – Script Block Logging (4104)	37
8.1.5	Eventos de login – fallido (4625) y exitoso (4624)	37
8.1.6	Eventos relacionados con servicios – Service Configuration (7040)	38
8.1.7	Eventos de cambios administrativos locales – User Account Created (4720)	38
8.1.8	Resumen de eventos controlados en Windows	40
8.2	Eventos controlados en Linux	40
8.2.1	CPU	40
8.2.2	Memoria	41
8.2.3	Red	42

8.2.4	Inicio o detención de un servicio	43
8.2.5	Revisión de procesos activos	44
9	Fase 7: Creación de visualizaciones y dashboard (Bonus)	46
9.1	Acceso al editor de Dashboards	46
9.2	Visualización 1 – Cantidad de eventos por host	46
9.3	Visualización 2 – Eventos Windows por canal	47
9.4	Visualización 3 – Eventos por agente	48
9.5	Visualización 4 – Métricas de CPU de la máquina Linux	49
9.6	Visualización 5 – Métricas de memoria de la máquina Linux	50
9.7	Visualización 6 – Uso de disco de la máquina Linux	51
9.8	Visualización 7 – Línea de tiempo de eventos recibidos	52
10	Fase 8: Análisis de seguridad	54

1 Descripción y objetivos

Descripción

Workshop del submódulo **Operaciones SOC y Monitoreo**: monitoreo de eventos y registros mediante un SIEM. En este ejercicio se implementa un entorno básico de monitoreo de seguridad con **Elasticsearch**, **Kibana**, **Winlogbeat** y **Metricbeat**, integrando una estación Windows y una Linux como fuentes de datos.

Objetivos

Evaluar de manera práctica los conocimientos del submódulo, integrando:

- Conceptos clave de monitoreo.
- Gestión de logs.
- Reglas para monitoreo de servicios.
- Generación de alertas a partir de eventos de seguridad.

Arquitectura del workshop

La arquitectura mínima del workshop está compuesta por:

- **Servidor SIEM / Elastic Stack**
 - **Elasticsearch**: almacenamiento, indexación y búsqueda de eventos.
 - **Kibana**: visualización, exploración y análisis de datos.
- **Máquina Windows** con **Winlogbeat** para recolección de eventos.
- **Máquina Linux** con **Metricbeat** para recolección de métricas.
- **Usuario analista** que accede a Kibana desde el navegador.

2 Fase 0: Preparativos y configuración de red

Se evidencia que la máquina virtual Kali se encuentra en modo NAT con la IP 192.168.161.131, como se ve en la siguiente imagen:

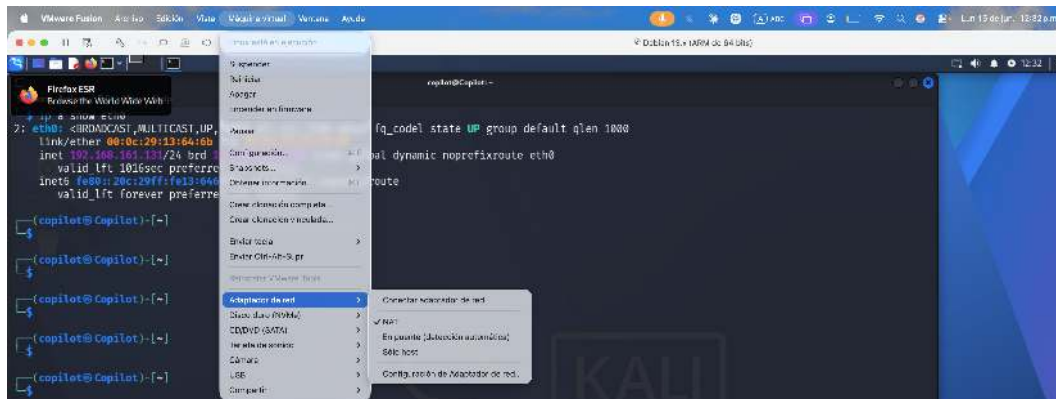


Figure 1: Kali Linux en modo NAT con IP 192.168.161.131.

La máquina virtual Windows también se encuentra en modo NAT y, en este punto, no es posible establecer comunicación entre ambas máquinas virtuales (ping fallido):

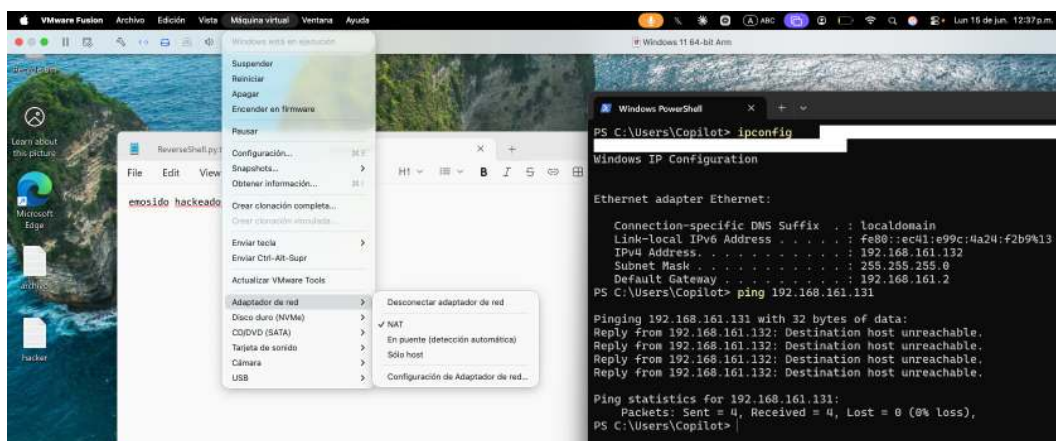


Figure 2: Windows en modo NAT: ping fallido entre VMs.

Posteriormente se configuran todas las máquinas virtuales en **Uso compartido de Internet** (NAT compartido con el host), como se evidencia en la máquina Ubuntu:

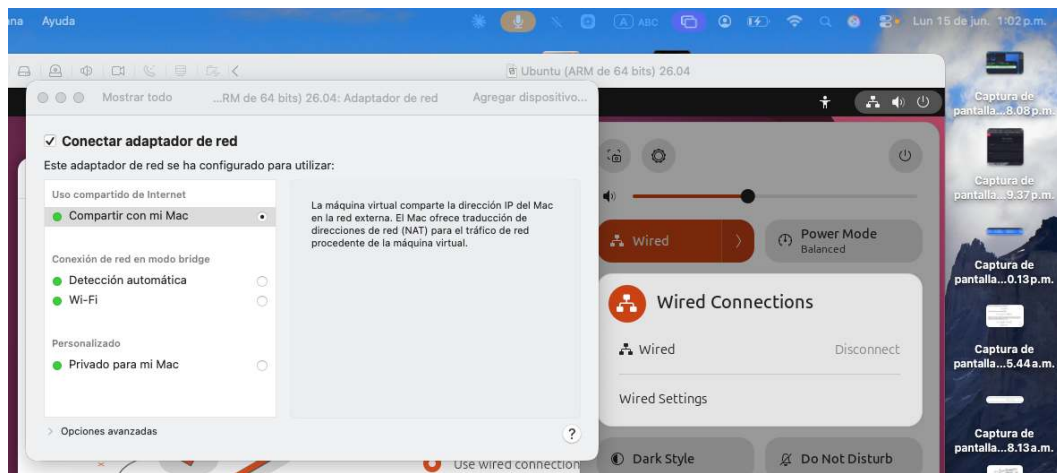


Figure 3: Ubuntu configurado en Uso compartido de Internet.

De este modo las máquinas virtuales quedan con las siguientes direcciones IP:

- Kali: 192.168.161.131
- Windows: 192.168.161.132
- Ubuntu: 192.168.161.133

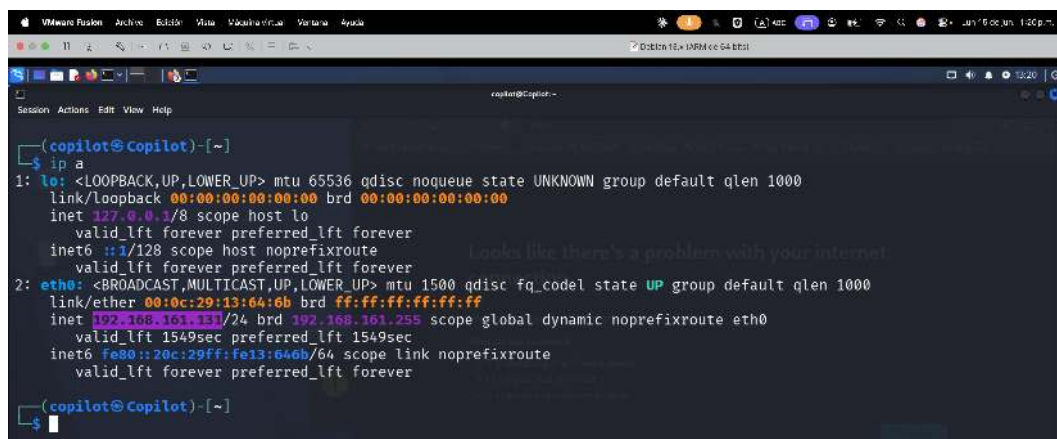


Figure 4: Verificación de IP – máquina 1.

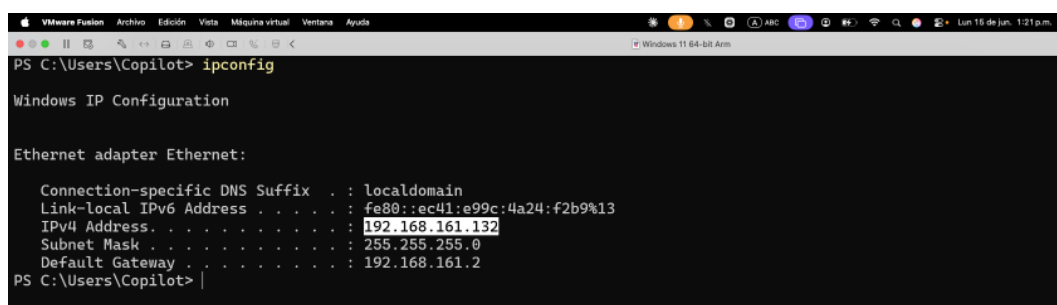
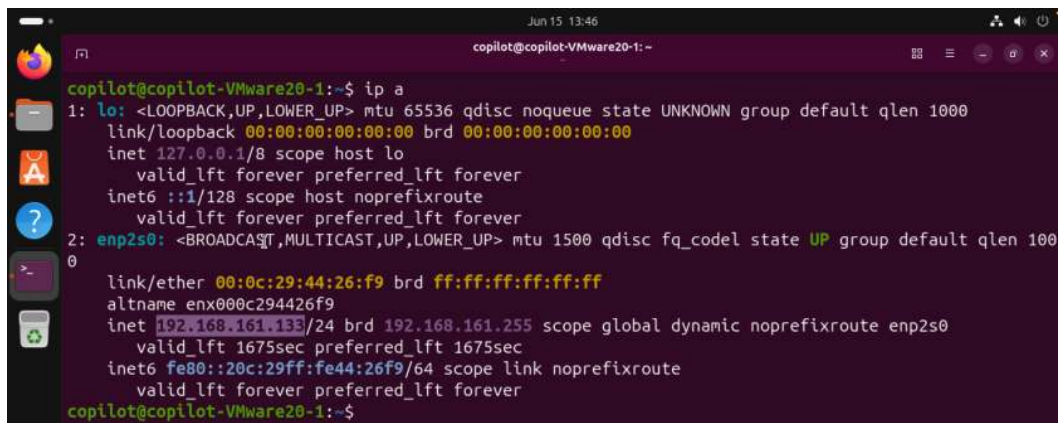


Figure 5: Verificación de IP – máquina 2.



```

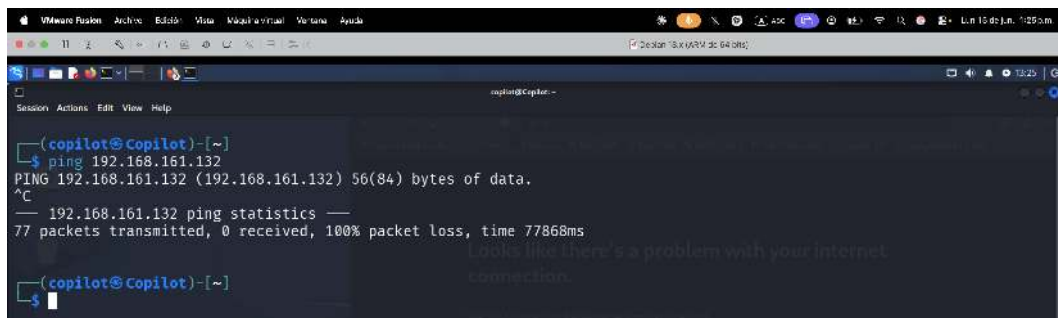
copilot@copilot-VMware20-1:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp2s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:44:26:f9 brd ff:ff:ff:ff:ff:ff
    altname enx00c294426f9
    inet 192.168.161.133/24 brd 192.168.161.255 scope global dynamic noprefixroute enp2s0
        valid_lft 1675sec preferred_lft 1675sec
    inet6 fe80::20c:29ff:fe44:26f9/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
copilot@copilot-VMware20-1:~$

```

Figure 6: Verificación de IP – máquina 3.

2.1 Problema de conectividad y solución

Se detecta que existe un problema para establecer conexión entre la máquina Windows y la máquina Kali:



```

(copilot@Copilot)~$ ping 192.168.161.132
PING 192.168.161.132 (192.168.161.132) 56(84) bytes of data:
^C
  192.168.161.132 ping statistics:
  77 packets transmitted, 0 received, 100% packet loss, time 77868ms
Looks like there's a problem with your internet connection.
(copilot@Copilot)~$

```

Figure 7: Ping fallido entre Kali y Windows antes de habilitar la regla ICMP.

Esto se debe a una regla del firewall que bloquea el tráfico ICMP en la máquina Windows. La solución consiste en abrir **Windows Defender Firewall with Advanced Security** y habilitar la regla **File and Printer Sharing (Echo Request – ICMPv4-In)**:

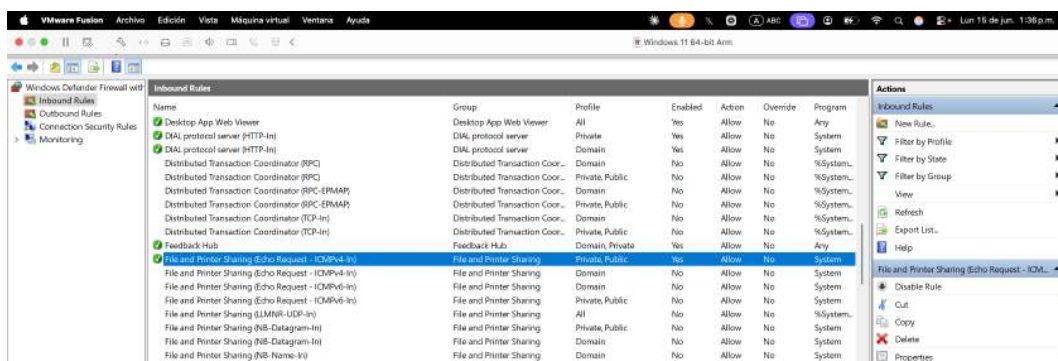
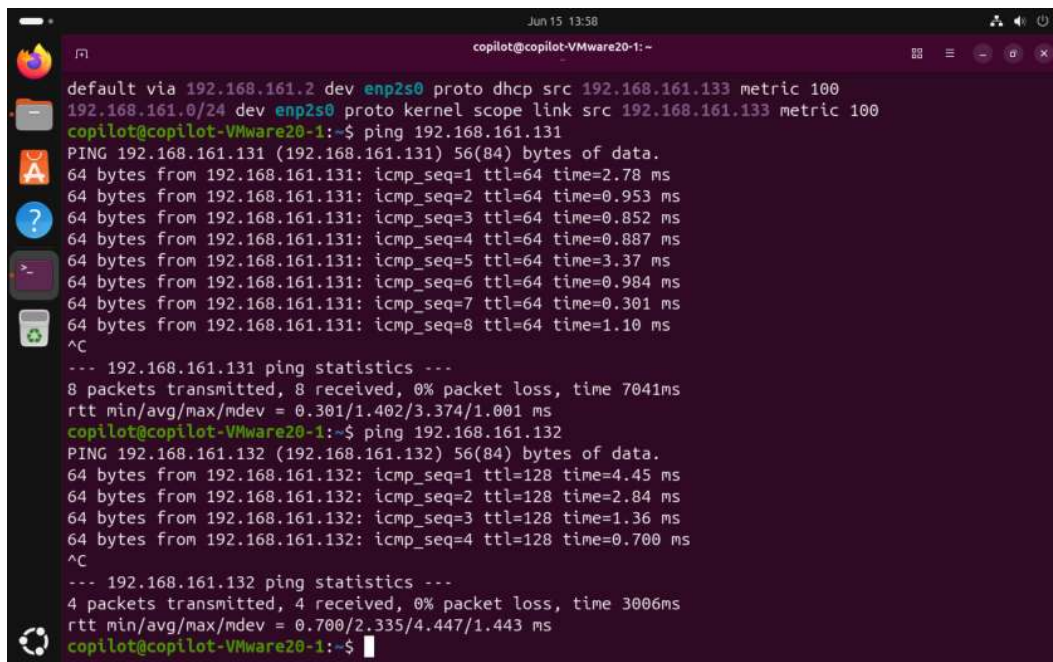


Figure 8: Habilitación de la regla ICMPv4-In en Windows Defender Firewall.

2.2 Verificación de conectividad entre las máquinas

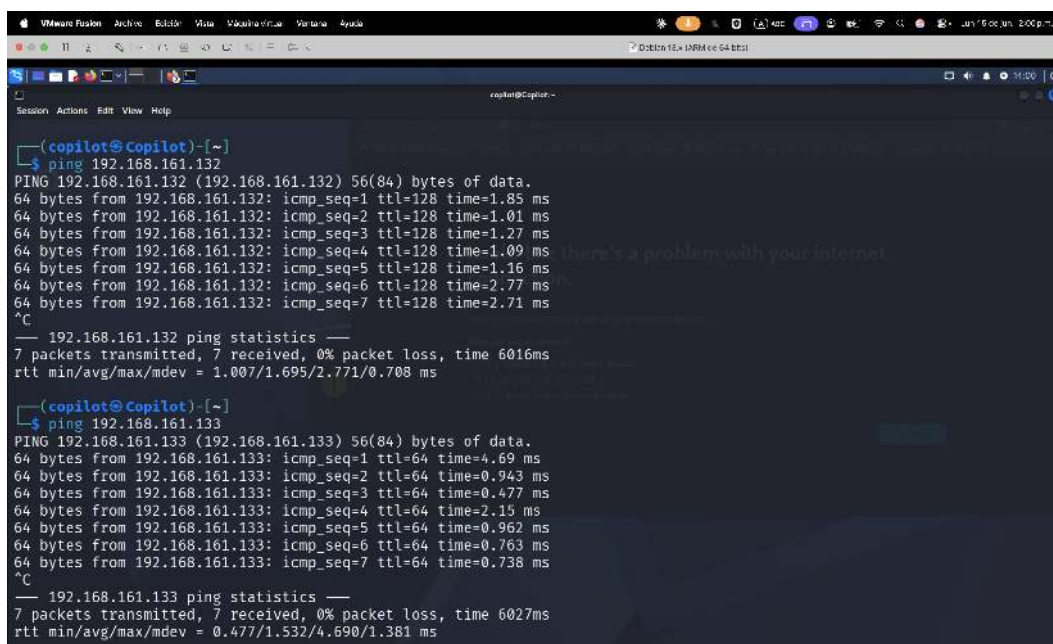
Ping exitoso Ubuntu → Kali y Ubuntu → Windows



```
Jun 15 13:58
copilot@copilot-VMware20-1: ~
default via 192.168.161.2 dev enp2s0 proto dhcp src 192.168.161.133 metric 100
192.168.161.0/24 dev enp2s0 proto kernel scope link src 192.168.161.133 metric 100
copilot@copilot-VMware20-1:~$ ping 192.168.161.131
PING 192.168.161.131 (192.168.161.131) 56(84) bytes of data:
64 bytes from 192.168.161.131: icmp_seq=1 ttl=64 time=2.78 ms
64 bytes from 192.168.161.131: icmp_seq=2 ttl=64 time=0.953 ms
64 bytes from 192.168.161.131: icmp_seq=3 ttl=64 time=0.852 ms
64 bytes from 192.168.161.131: icmp_seq=4 ttl=64 time=0.887 ms
64 bytes from 192.168.161.131: icmp_seq=5 ttl=64 time=3.37 ms
64 bytes from 192.168.161.131: icmp_seq=6 ttl=64 time=0.984 ms
64 bytes from 192.168.161.131: icmp_seq=7 ttl=64 time=0.301 ms
64 bytes from 192.168.161.131: icmp_seq=8 ttl=64 time=1.10 ms
^C
--- 192.168.161.131 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7041ms
rtt min/avg/max/mdev = 0.301/1.402/3.374/1.001 ms
copilot@copilot-VMware20-1:~$ ping 192.168.161.132
PING 192.168.161.132 (192.168.161.132) 56(84) bytes of data:
64 bytes from 192.168.161.132: icmp_seq=1 ttl=128 time=4.45 ms
64 bytes from 192.168.161.132: icmp_seq=2 ttl=128 time=2.84 ms
64 bytes from 192.168.161.132: icmp_seq=3 ttl=128 time=1.36 ms
64 bytes from 192.168.161.132: icmp_seq=4 ttl=128 time=0.700 ms
^C
--- 192.168.161.132 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 0.700/2.335/4.447/1.443 ms
copilot@copilot-VMware20-1:~$
```

Figure 9: Conectividad establecida desde Ubuntu hacia Kali y Windows.

Ping exitoso Kali → Ubuntu y Kali → Windows



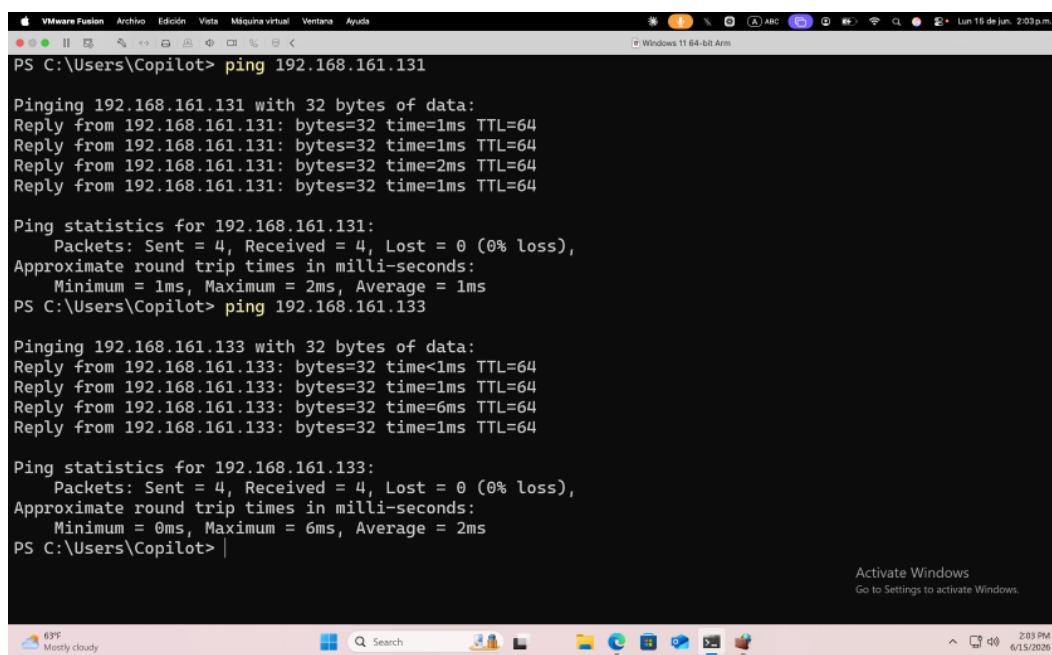
```
VMware Fusion
copilot@Copilot: ~
$ ping 192.168.161.132
PING 192.168.161.132 (192.168.161.132) 56(84) bytes of data:
64 bytes from 192.168.161.132: icmp_seq=1 ttl=128 time=1.85 ms
64 bytes from 192.168.161.132: icmp_seq=2 ttl=128 time=1.01 ms
64 bytes from 192.168.161.132: icmp_seq=3 ttl=128 time=1.27 ms
64 bytes from 192.168.161.132: icmp_seq=4 ttl=128 time=1.09 ms
64 bytes from 192.168.161.132: icmp_seq=5 ttl=128 time=1.16 ms
64 bytes from 192.168.161.132: icmp_seq=6 ttl=128 time=2.77 ms
64 bytes from 192.168.161.132: icmp_seq=7 ttl=128 time=2.71 ms
^C
--- 192.168.161.132 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6016ms
rtt min/avg/max/mdev = 1.007/1.695/2.771/0.708 ms

copilot@Copilot: ~
$ ping 192.168.161.133
PING 192.168.161.133 (192.168.161.133) 56(84) bytes of data:
64 bytes from 192.168.161.133: icmp_seq=1 ttl=64 time=4.69 ms
64 bytes from 192.168.161.133: icmp_seq=2 ttl=64 time=0.943 ms
64 bytes from 192.168.161.133: icmp_seq=3 ttl=64 time=0.477 ms
64 bytes from 192.168.161.133: icmp_seq=4 ttl=64 time=2.15 ms
64 bytes from 192.168.161.133: icmp_seq=5 ttl=64 time=0.962 ms
64 bytes from 192.168.161.133: icmp_seq=6 ttl=64 time=0.763 ms
64 bytes from 192.168.161.133: icmp_seq=7 ttl=64 time=0.738 ms
^C
--- 192.168.161.133 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6027ms
rtt min/avg/max/mdev = 0.477/1.532/4.690/1.381 ms
```

Figure 10: Conectividad establecida desde Kali hacia Ubuntu y Windows.

Ping exitoso Windows → Ubuntu y Windows → Kali

Después de habilitar la regla del firewall en la máquina Windows, se establece conexión entre Windows y las demás VMs:



```
PS C:\Users\Copilot> ping 192.168.161.131

Pinging 192.168.161.131 with 32 bytes of data:
Reply from 192.168.161.131: bytes=32 time=1ms TTL=64
Reply from 192.168.161.131: bytes=32 time=1ms TTL=64
Reply from 192.168.161.131: bytes=32 time=2ms TTL=64
Reply from 192.168.161.131: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.161.131:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 2ms, Average = 1ms
PS C:\Users\Copilot> ping 192.168.161.133

Pinging 192.168.161.133 with 32 bytes of data:
Reply from 192.168.161.133: bytes=32 time<1ms TTL=64
Reply from 192.168.161.133: bytes=32 time=1ms TTL=64
Reply from 192.168.161.133: bytes=32 time=6ms TTL=64
Reply from 192.168.161.133: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.161.133:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 6ms, Average = 2ms
PS C:\Users\Copilot> |
```

Figure 11: Conectividad establecida desde Windows hacia Ubuntu y Kali.

3 Fase 1: Diseño del entorno

3.1 Resumen del diseño

A continuación se documentan los elementos básicos del diseño del workshop: direcciones IP, roles, puertos, flujo de datos, forma de acceso a Kibana y riesgos básicos identificados.

Table 1: Diseño del entorno del workshop

Elemento	Detalle		
Dirección IP de cada máquina	Máquina	IP	Sistema operativo
	Kali	192.168.161.131	Kali Linux
	Windows	192.168.161.132	Windows 10
	Ubuntu	192.168.161.133	Ubuntu 20.04
Rol de cada sistema	Kali	Servidor SIEM / Elastic Stack (Elasticsearch + Kibana).	
	Windows	Agente encargado de recolectar logs del sistema operativo Windows (Winlogbeat) y enviarlos al servidor SIEM.	
	Ubuntu	Agente para recolectar métricas del sistema Linux (Metricbeat) y enviarlas a Elasticsearch.	
Puertos requeridos para la comunicación	Puerto	Protocolo	Uso
	9200	HTTPS/HTTP	API REST de Elasticsearch (ingesta de Beats y consultas).
	5601	HTTPS/HTTP	Interfaz web de Kibana para el analista.
	ICMP	–	Diagnóstico de conectividad entre VMs (ping).
Flujo de datos entre los agentes y Elasticsearch	- Winlogbeat (Windows) .132 → Elasticsearch .131:9200. - Metricbeat (Ubuntu) .133 → Elasticsearch .131:9200. - Kibana (.131:5601) ↔ Elasticsearch (127.0.0.1:9200). - Analista / navegador → Kibana (.131:5601).		
Forma de acceso a Kibana	Desde el navegador web del analista mediante la URL <code>http://192.168.161.131:5601</code> , autenticándose con el usuario <code>elastic</code> y la contraseña generada durante la instalación de Elasticsearch.		
Riesgos básicos del workshop	Exposición de servicios: Elasticsearch (9200) y Kibana (5601) escuchando en la red del laboratorio sin restricciones de IP. Credenciales débiles o por defecto: riesgo de accesos no autorizados si no se cambia la contraseña inicial del usuario <code>elastic</code>. Falta de cifrado en tránsito: si se usa HTTP en lugar de HTTPS, los eventos y credenciales viajan en texto claro. Falta de segmentación: todas las VMs comparten la misma subred (192.168.161.0/24). Superficie de ataque del agente: Winlogbeat/Metricbeat corren con privilegios elevados. Persistencia de logs: sin política de retención, los índices pueden agotar el disco del servidor SIEM.		

4 Fase 2: Instalación de Elasticsearch y Kibana

4.1 Instalación de Elasticsearch en Kali Linux

Cuando intentamos instalar Elasticsearch en la máquina Kali, nos encontramos con el siguiente error:

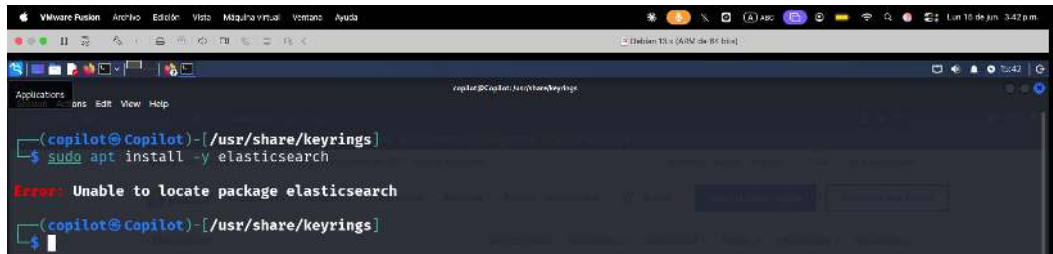


Figure 12: Error inicial al intentar instalar Elasticsearch sin el repositorio configurado.

Esto se debe a que el repositorio de Elasticsearch no se encuentra configurado en la máquina Kali, por lo que debemos agregarlo para poder instalar Elasticsearch. Para ello ejecutamos los siguientes comandos:

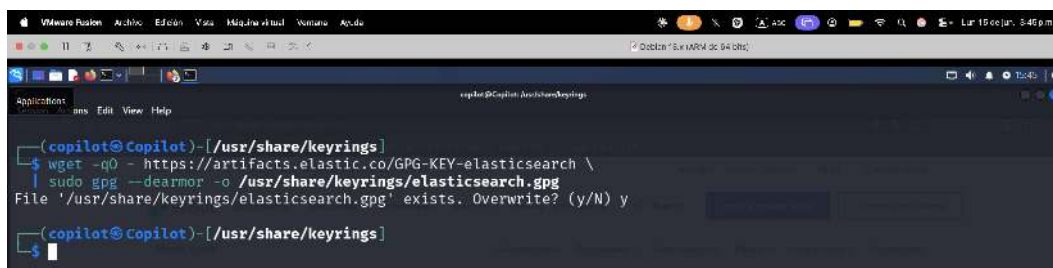


Figure 13: Configuración del repositorio de Elastic.

Primero descargamos la llave GPG del repositorio de Elasticsearch con el siguiente comando:

```
1 wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch
```

Se convierte en binario para que pueda ser proporcionada a `apt-key` con el siguiente comando, y se termina guardando en el directorio de claves de `apt` para que pueda ser utilizada por `apt-key`:

```
1 gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg
```

Luego se agrega el repositorio de Elasticsearch a la lista de repositorios de `apt` con el siguiente comando:

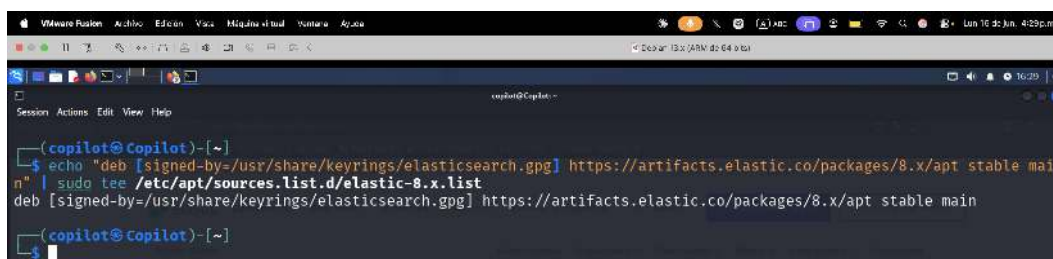


Figure 14: Adición del repositorio Elastic 8.x a las fuentes de APT.

Por último usamos nuevamente `apt update` para actualizar la lista de paquetes disponibles y observamos que se actualizó correctamente:

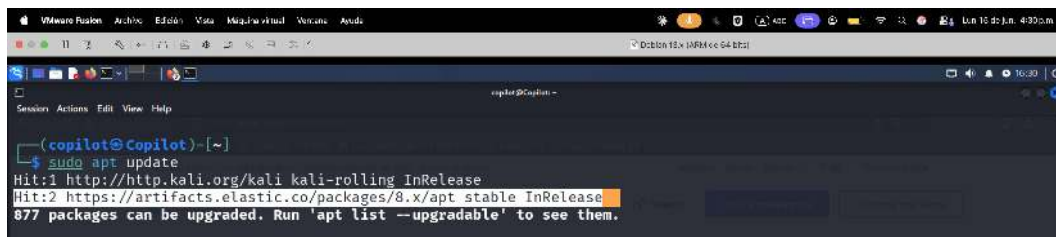


Figure 15: Actualización exitosa de la lista de paquetes con el nuevo repositorio.

Y podemos instalar Elasticsearch con el siguiente comando:

```
1 sudo apt install elasticsearch
```

Observamos que se nos entrega una credencial de superusuario para **elastic**, la cual es importante guardar para poder acceder a Elasticsearch posteriormente:

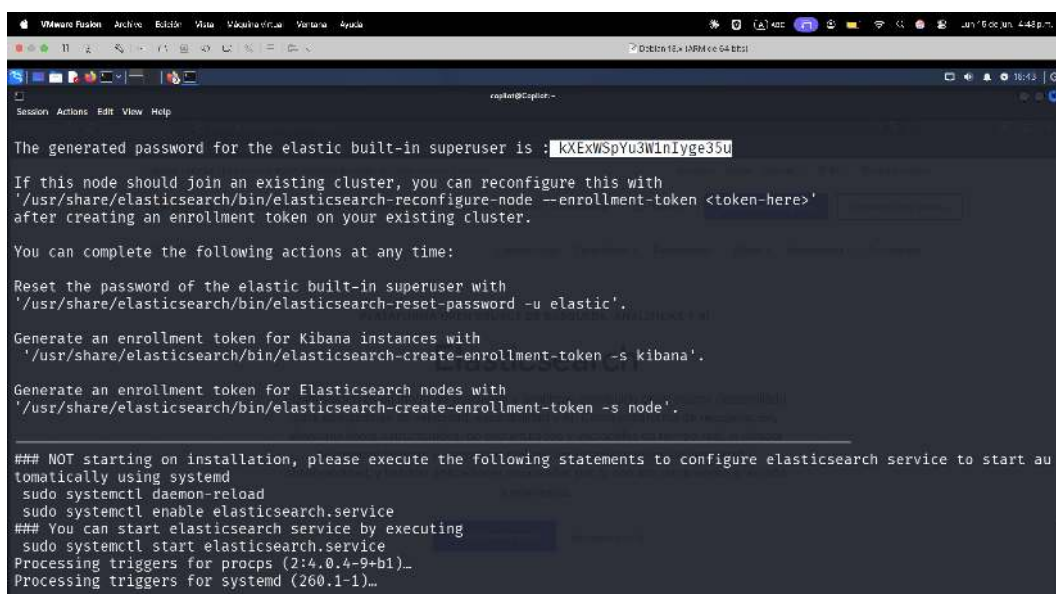
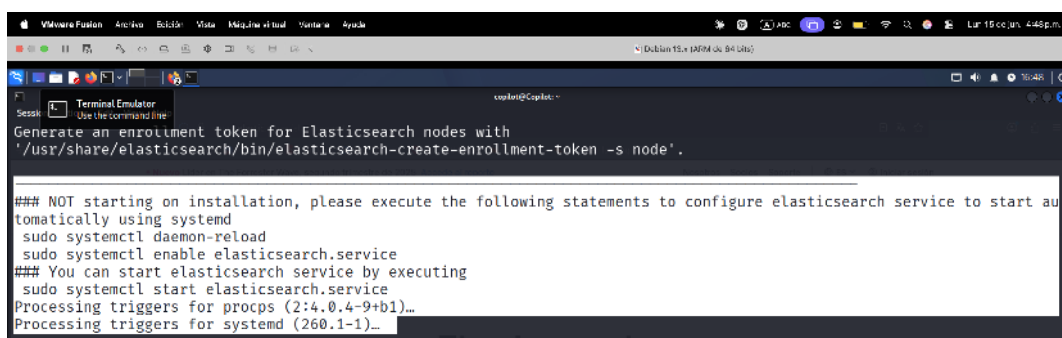


Figure 16: Credenciales del superusuario **elastic** generadas durante la instalación.

Además, se entregan instrucciones para ejecutar Elasticsearch:



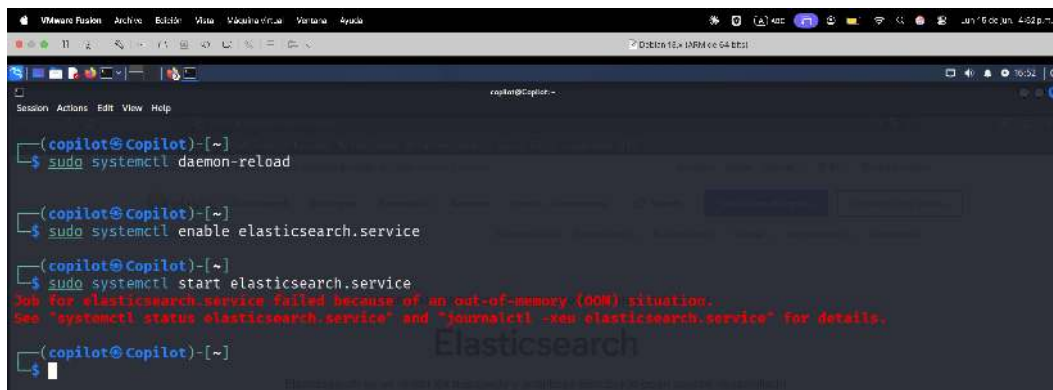


Figure 18: Error de inicio del servicio por insuficiencia de memoria (OOM).

Al parecer es debido a un problema de insuficiencia de memoria, por lo que se debe aumentar la memoria RAM de la máquina virtual Kali a **4 GB** para poder ejecutar Elasticsearch correctamente, como se ve en la siguiente imagen:

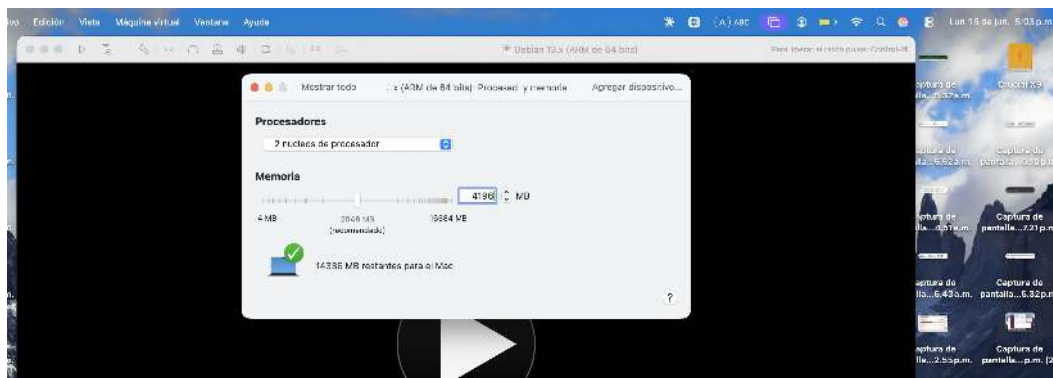


Figure 19: Aumento de la memoria RAM de la VM Kali a 4GB.

Y encontramos que el servicio se encuentra activo y corriendo correctamente:

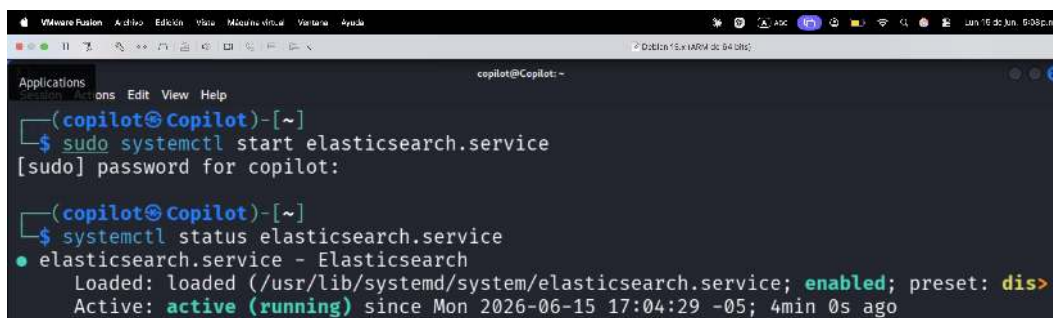


Figure 20: Servicio de Elasticsearch active (running).

4.2 Instalación de Kibana en Kali Linux

Dado que Kibana se encuentra en el mismo repositorio que Elasticsearch, no es necesario agregar un nuevo repositorio para instalar Kibana, por lo que simplemente ejecutamos el siguiente comando:

```
1 sudo apt install kibana
```

Configuramos el puerto y el host de Kibana para que pueda ser accedido desde otras máquinas virtuales, editando el archivo de configuración de Kibana con el siguiente comando:

```
1 sudo nano /etc/kibana/kibana.yml
```

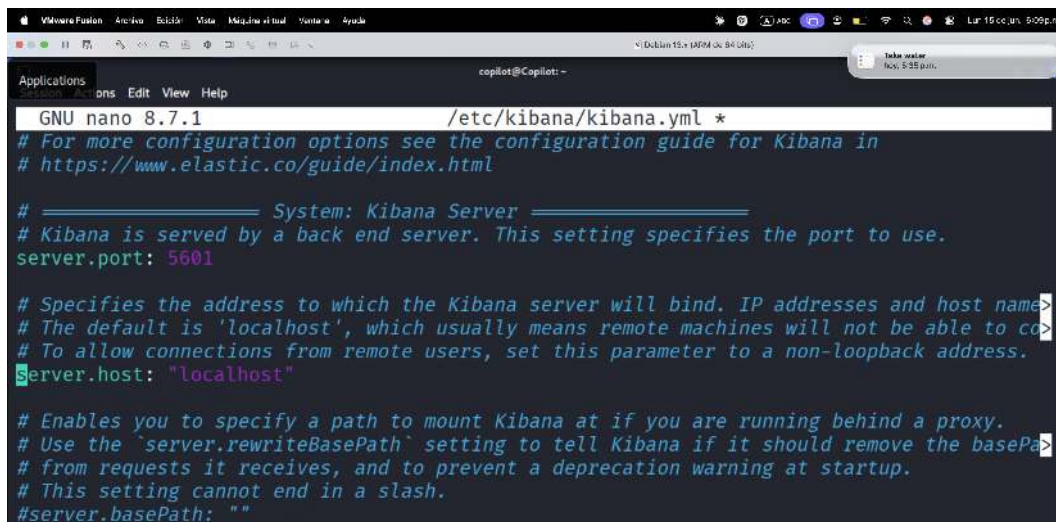


Figure 21: Edición de kibana.yml (host y puerto).

Y arrancamos el servicio de Kibana con los siguientes comandos:

```

1 sudo systemctl daemon-reload
2 sudo systemctl enable kibana --now
  
```

Observamos que el servicio se encuentra activo y corriendo correctamente:

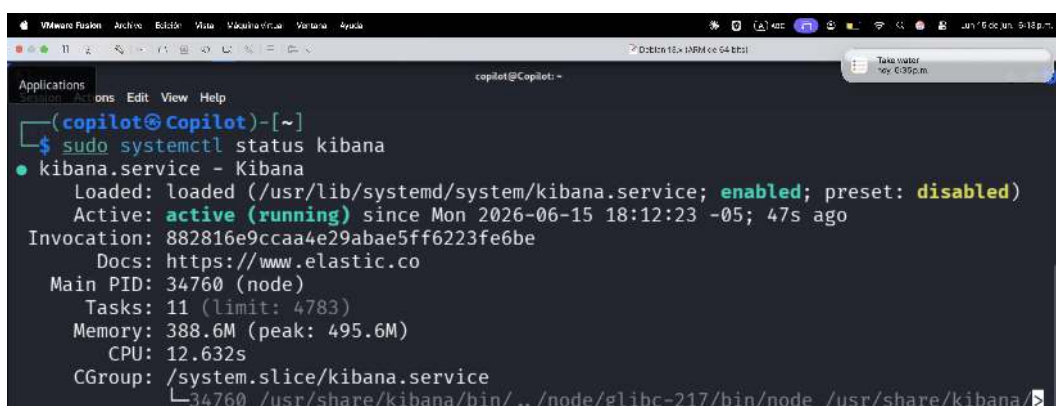


Figure 22: Servicio de Kibana active (running).

Y ejecutando el siguiente comando:

```

1 sudo journalctl -u kibana -n 30 --no-pager
  
```

Observamos que podemos acceder desde el navegador a Elasticsearch y Kibana desde la máquina Kali:

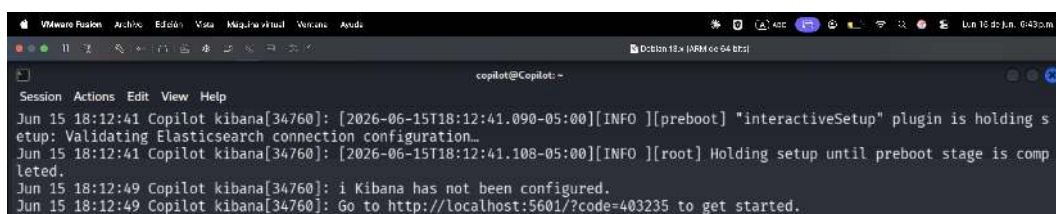


Figure 23: Acceso a Kibana desde el navegador en la VM Kali.

Ahora nos es solicitado un token de autenticación para poder acceder a Kibana. Para obtenerlo, ejecutamos el siguiente comando:

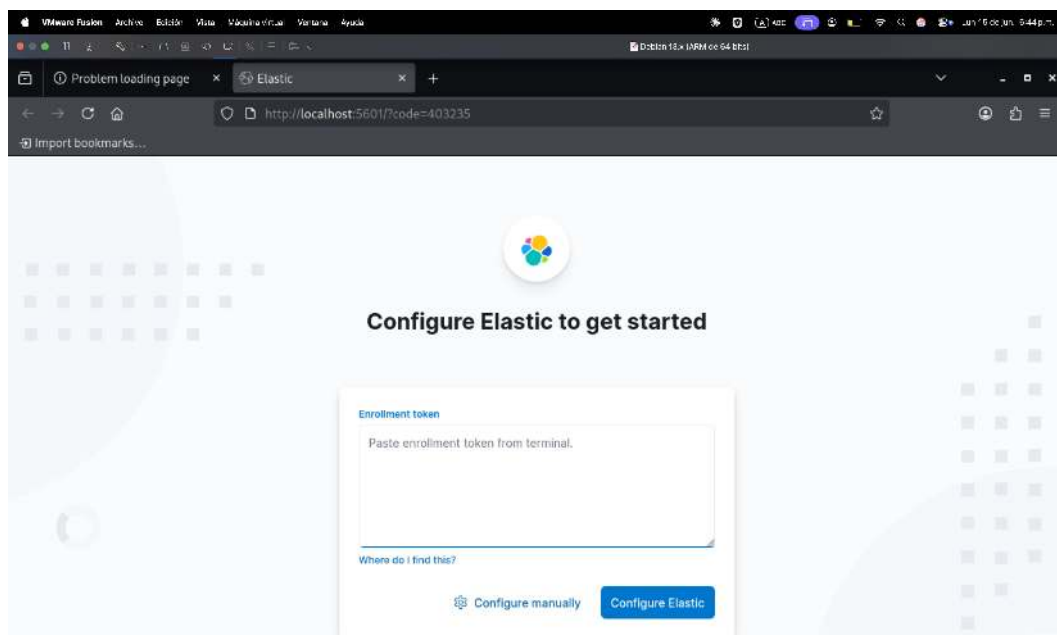


Figure 24: Pantalla de Kibana solicitando el token de enrolamiento.

```
1 sudo /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana
```

Lo cual nos proporciona el token de autenticación y las credenciales del usuario `elastic` para poder acceder a Kibana:

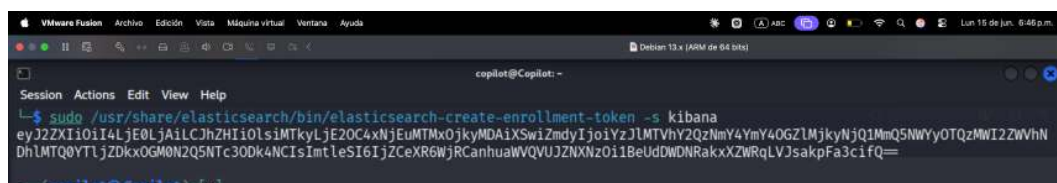


Figure 25: Token de enrolamiento generado.

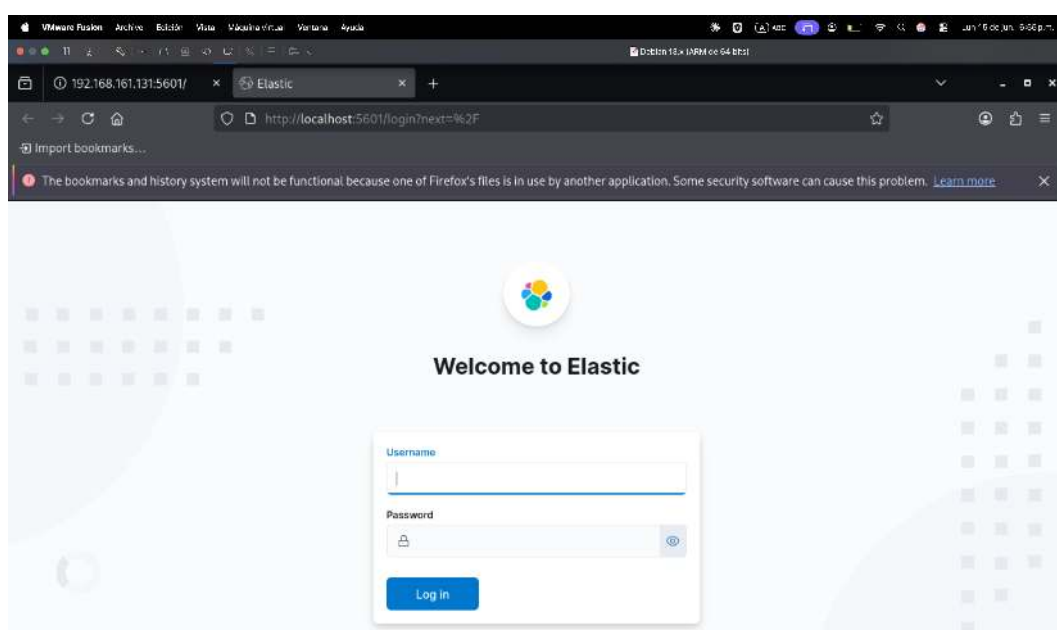


Figure 26: Pegado del token en Kibana e ingreso del código de verificación.

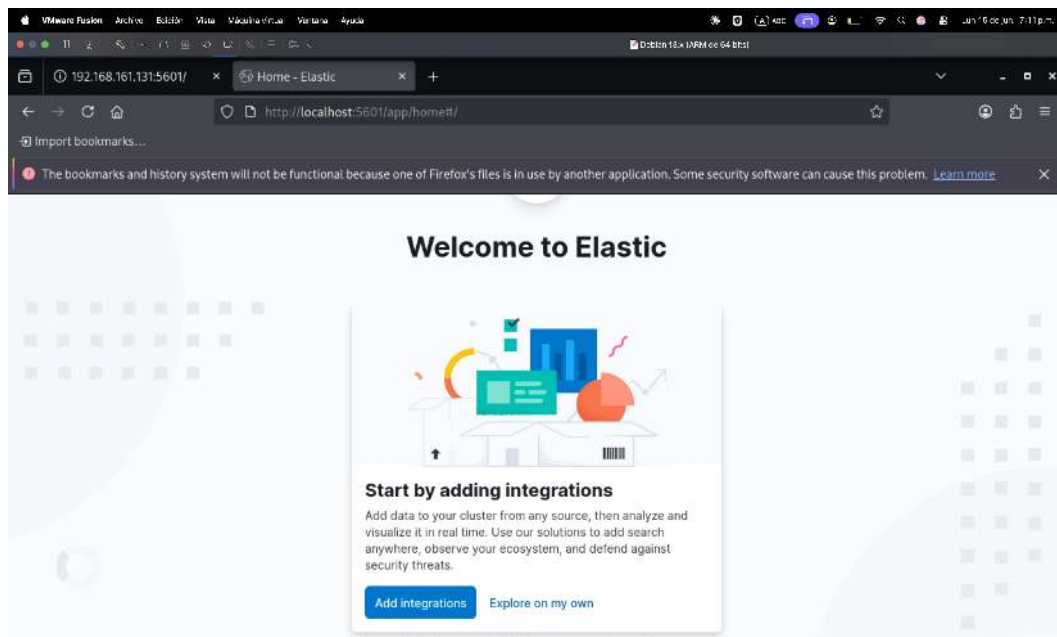


Figure 27: Login exitoso con el usuario elastic.

4.3 Verificación de que Elasticsearch recibe y responde consultas

Verificamos además que Elasticsearch responde consultas correctamente, ejecutando las siguientes consultas desde el **Dev Tools** de Kibana:

```
1 GET / // Información general del cluster
2 GET /_cluster/health // Salud del cluster
3 GET /_cat/_search // Verificación del motor de búsqueda
```

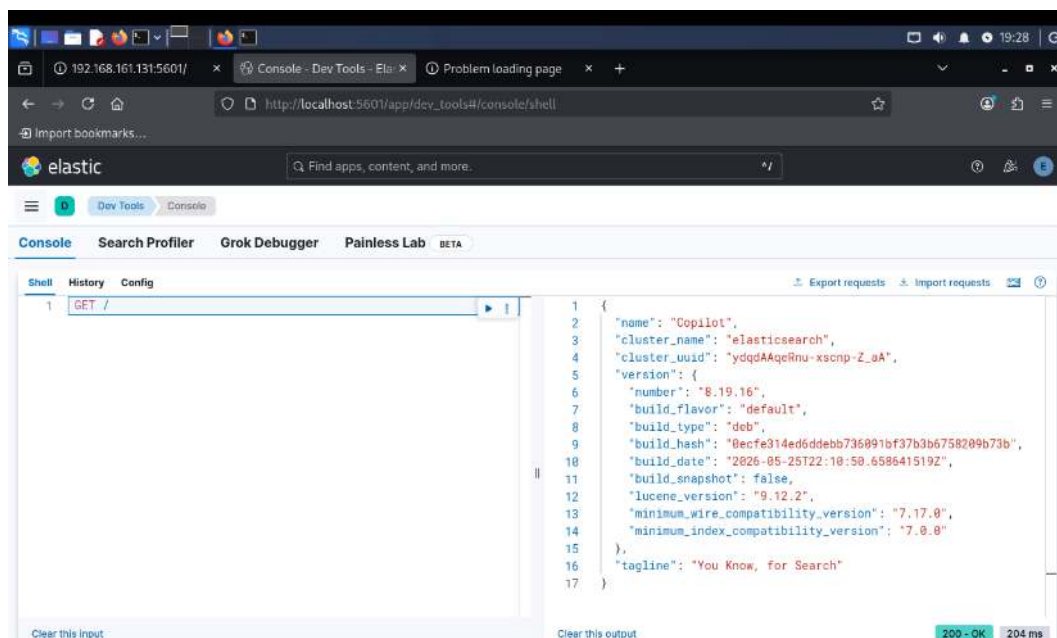


Figure 28: Respuesta a GET / con información del cluster.

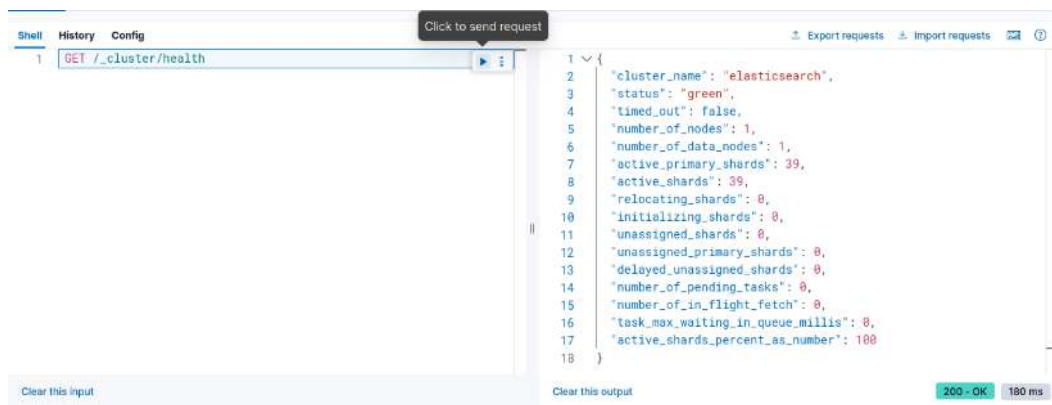


Figure 29: Respuesta a GET /_cluster/health.

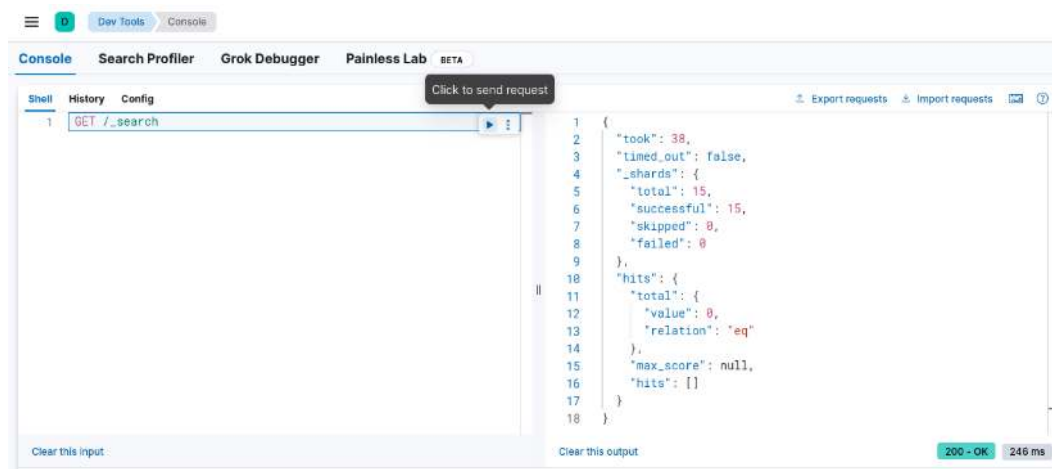


Figure 30: Verificación de que el motor de búsqueda responde.

5 Fase 3: Integración de eventos Windows con Winlogbeat

5.1 Instalación y configuración inicial

Procedemos a descargar e instalar Winlogbeat 8.19.16 en la máquina Windows:

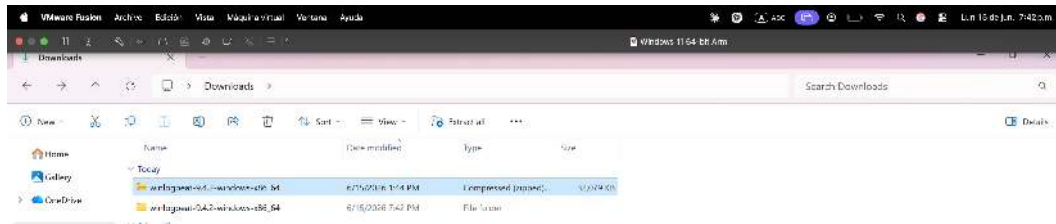


Figure 31: Descarga del paquete MSI de Winlogbeat 8.19.16.

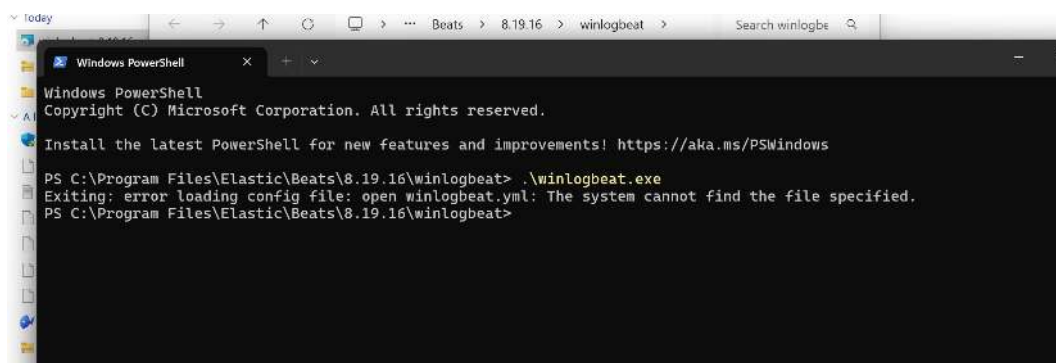


Figure 32: Primer intento de ejecución: error por falta del archivo winlogbeat.yml.

Esta última imagen significa que tenemos que configurar el archivo de configuración de Winlogbeat para que pueda enviar los logs a Elasticsearch. Configuramos las credenciales obtenidas en la Fase 2:

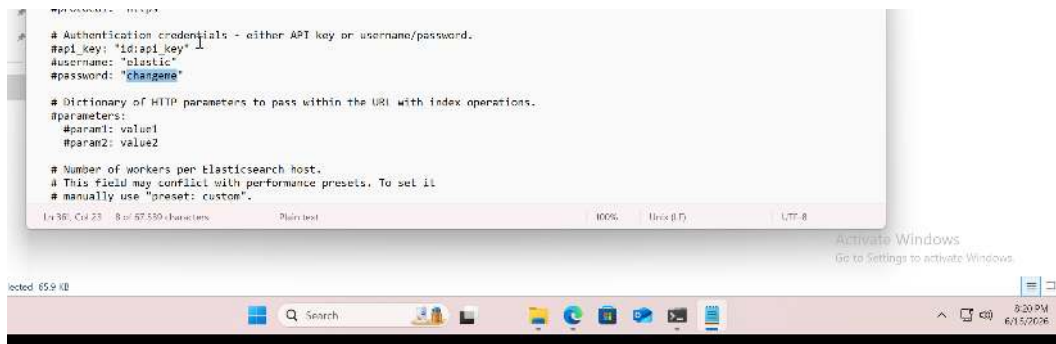


Figure 33: Configuración de credenciales elastic en winlogbeat.yml.

Adicionalmente, configuramos el host remoto y desactivamos la validación SSL para que pueda enviar los logs a Elasticsearch sin problemas. También configuramos la sección setup.kibana:

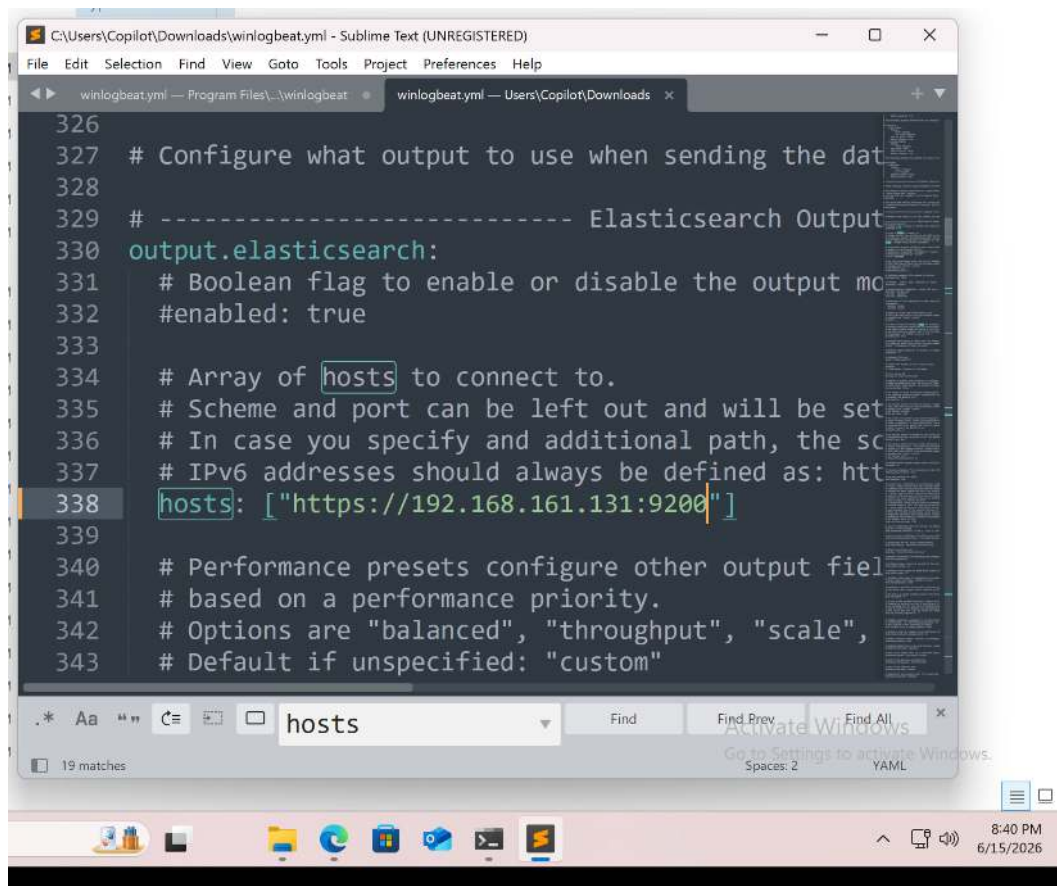


Figure 34: Configuración de output.elasticsearch con `ssl.verification_mode: none`.

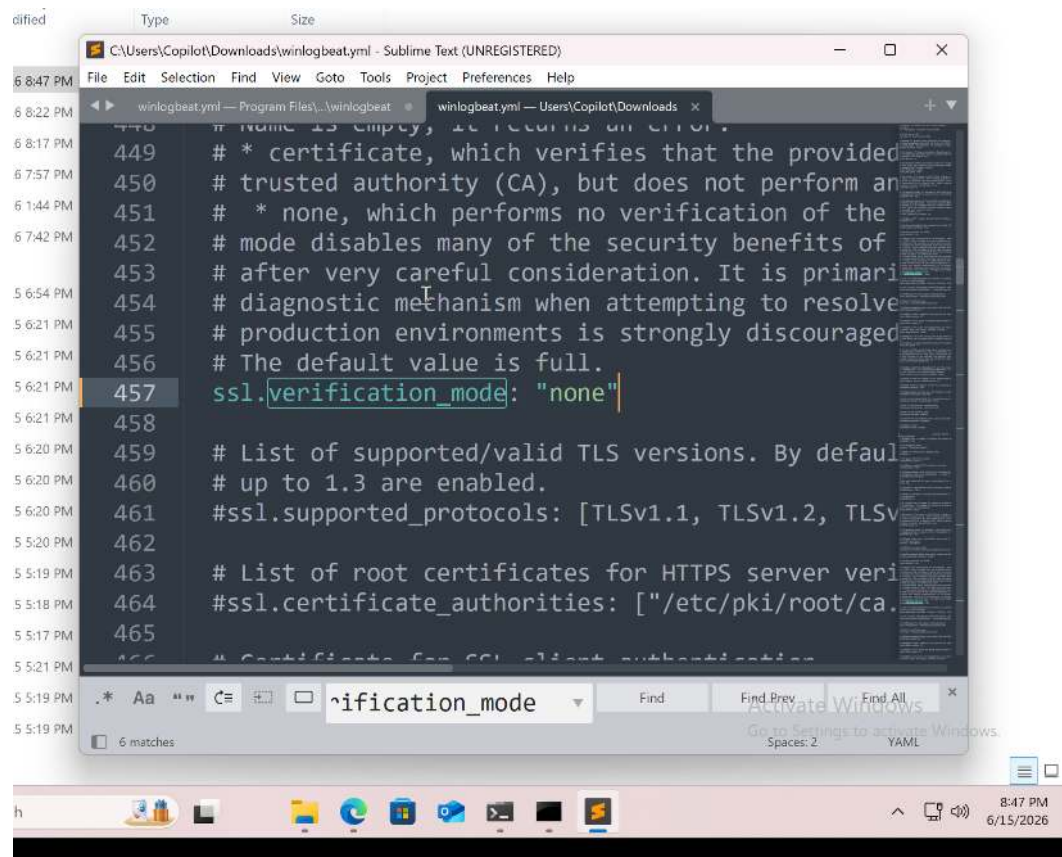


Figure 35: Configuración de setup.kibana.

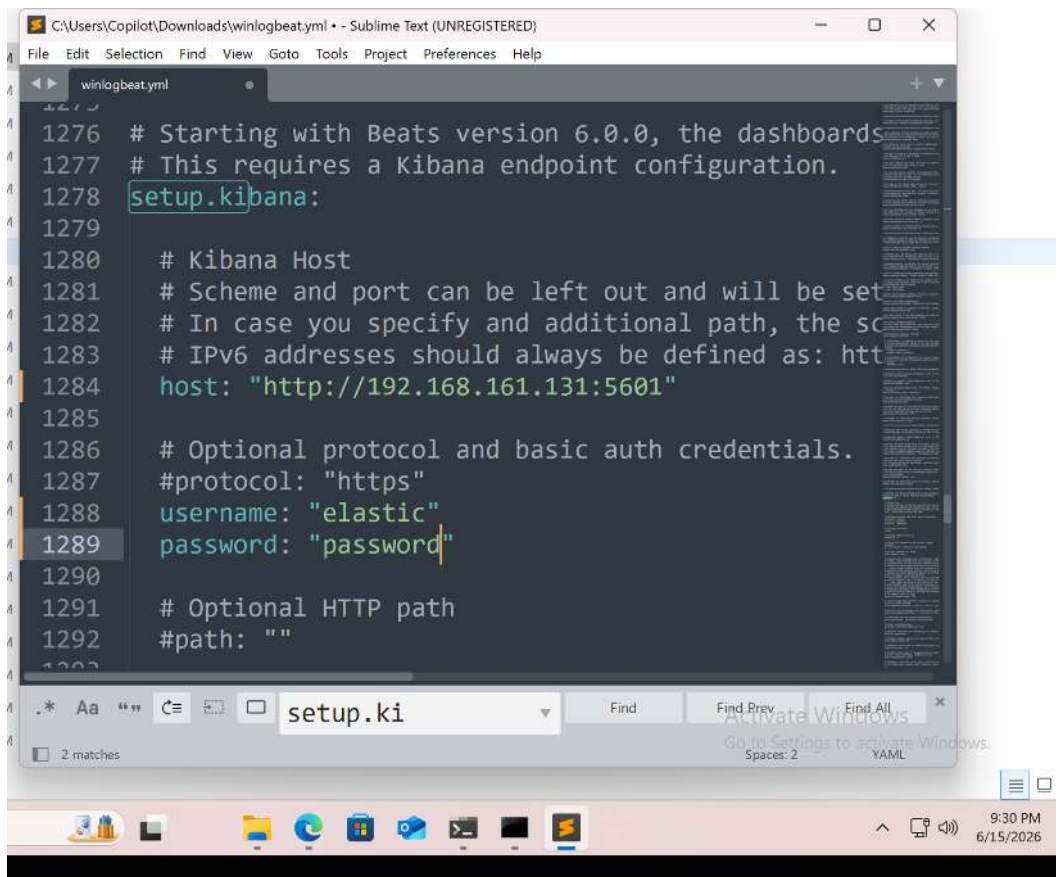


Figure 36: Validación de la configuración con `winlogbeat.exe test config`.

5.2 Problema de conexión a Kibana y solución

Encontramos el siguiente problema: se rechaza la conexión a Kibana desde la máquina Windows. Esto se debe a que el firewall de Windows bloquea inicialmente las conexiones, pero la causa principal es que Kibana solo escucha en localhost por defecto:

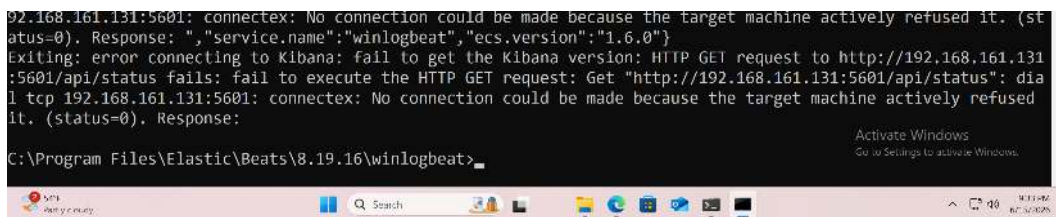


Figure 37: Error: conexión a Kibana rechazada (localhost:5601).

```

PS C:\Users\Copilot> Test-NetConnection -ComputerName 192.168.161.131 -Port 5601
WARNING: TCP connect to (192.168.161.131 : 5601) failed

ComputerName      : 192.168.161.131
RemoteAddress     : 192.168.161.131
RemotePort        : 5601
InterfaceAlias    : Ethernet
SourceAddress     : 192.168.161.132
PingSucceeded     : True
PingReplyDetails (RTT) : 0 ms
TcpTestSucceeded  : False

PS C:\Users\Copilot>

```

Figure 38: Detalle del error reportado por Winlogbeat al intentar contactar a Kibana.

La solución consiste en editar el archivo de configuración de Kibana en la máquina Kali:

```

GNU nano 8.7.1 /etc/kibana/kibana.yml
# For more configuration options see the configuration guide for Kibana in
# https://www.elastic.co/guide/index.html

# System: Kibana Server
# Kibana is served by a back end server. This setting specifies the port to use.
server.port: 5601

# Specifies the address to which the Kibana server will bind. IP addresses and host names are both valid values.
# The default is 'localhost', which usually means remote machines will not be able to connect.
# To allow connections from remote users, set this parameter to a non-loopback address.
server.host: 0.0.0.0

```

Figure 39: Edición de /etc/kibana/kibana.yml para que escuche en todas las interfaces.

Colocamos el valor `server.host: "0.0.0.0"` para que Kibana escuche en todas las interfaces de red, y reiniciamos el servicio de Kibana para que los cambios surtan efecto.

Vemos que Winlogbeat ahora se ejecuta correctamente, cargan los dashboards de Kibana y se pueden visualizar los logs enviados desde la máquina Windows a Elasticsearch, lo cual indica que la instalación de Winlogbeat fue exitosa y se encuentra enviando los logs a Elasticsearch correctamente:

```

C:\Program Files\Elastic\Beats\8.19.16\winlogbeat>winlogbeat.exe -e
{"log.level":"info","@timestamp":"2026-06-15T21:00:25.467-0500","log.origin":{"function":"github.com/elastic/beats/v7/libbeat/cmd/instance.(*Beat).configure","file.name":"instance/beat.go","file.line":834},"message":"Home path: [C:\\Program Files\\Elastic\\Beats\\8.19.16\\winlogbeat] Config path: [C:\\Program Files\\Elastic\\Beats\\8.19.16\\winlogbeat] Data path: [C:\\Program Files\\Elastic\\Beats\\8.19.16\\winlogbeat\\data] Logs path: [C:\\Program Files\\Elastic\\Beats\\8.19.16\\winlogbeat\\logs]","service.name":"winlogbeat","ecs.version":"1.6.0"}
{"log.level":"info","@timestamp":"2026-06-15T21:00:25.469-0500","log.origin":{"function":"github.com/elastic/beats/v7/libbeat/cmd/instance.(*Beat).configure","file.name":"instance/beat.go","file.line":842},"message":"Beat ID: b6abd78b-50b4-4d95-94f1-6e974c26fdb9","service.name":"winlogbeat","ecs.version":"1.6.0"}
{"log.level":"info","@timestamp":"2026-06-15T21:00:25.483-0500","log.logger":"beat","log.origin":{"function":"github.com/elastic/beats/v7/libbeat/cmd/instance.(*Beat).createBeater","file.name":"instance/beat.go","file.line":331},"message":"Setup Beat: winlogbeat; Version: 8.19.16","service.name":"winlogbeat","ecs.version":"1.6.0"}
{"log.level":"info","@timestamp":"2026-06-15T21:00:25.483-0500","log.logger":"beat","log.origin":{"function":"github.com/elastic/beats/v7/libbeat/cmd/instance.(*Beat).logSystemInfo","file.name":"instance/beat.go","file.line":1392},"message":"Beat info","service.name":"winlogbeat","system.info":{"beat":{"path":{"config":"C:\\Program Files\\Elastic\\Beats\\8.19.16\\winlogbeat","data":"C:\\Program Files\\Elastic\\Beats\\8.19.16\\winlogbeat\\data","home":"C:\\Program Files\\Elastic\\Beats\\8.19.16\\winlogbeat","logs":"C:\\Program Files\\Elastic\\Beats\\8.19.16\\winlogbeat\\logs"},"type":"winlogbeat","uuid":"b6abd78b-50b4-4d95-94f1-6e974c26fdb9"},"ecs.version":"1.6.0"}}

```

Figure 40: Ejecución exitosa de winlogbeat setup -e.

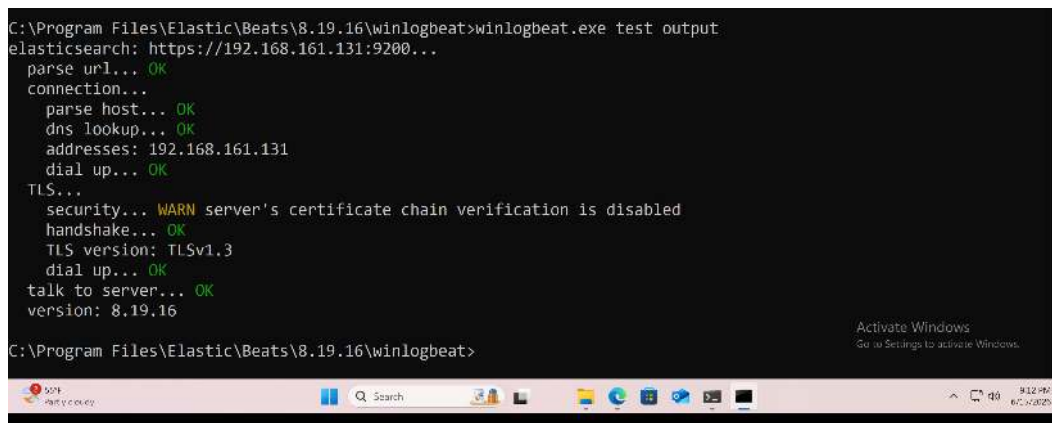


Figure 41: Carga de dashboards y plantillas en Kibana.

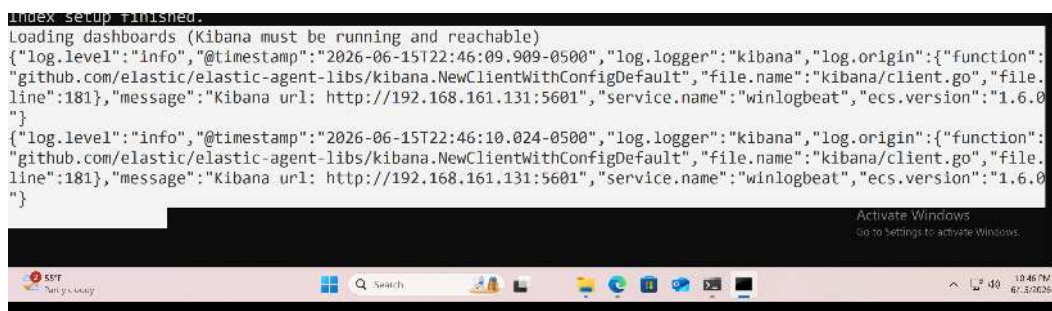


Figure 42: Confirmación de conexión y carga de plantillas en Elasticsearch.

5.3 Inicio del servicio Winlogbeat

Iniciamos el servicio de Winlogbeat para que comience a enviar los logs a Elasticsearch, y observamos que se encuentra activo y corriendo correctamente:

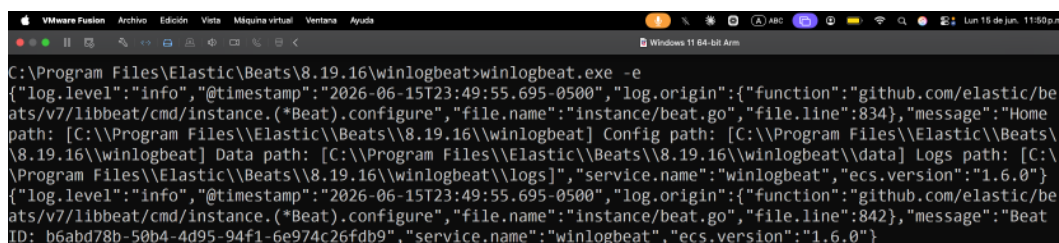


Figure 43: Instalación y arranque de Winlogbeat como servicio de Windows.

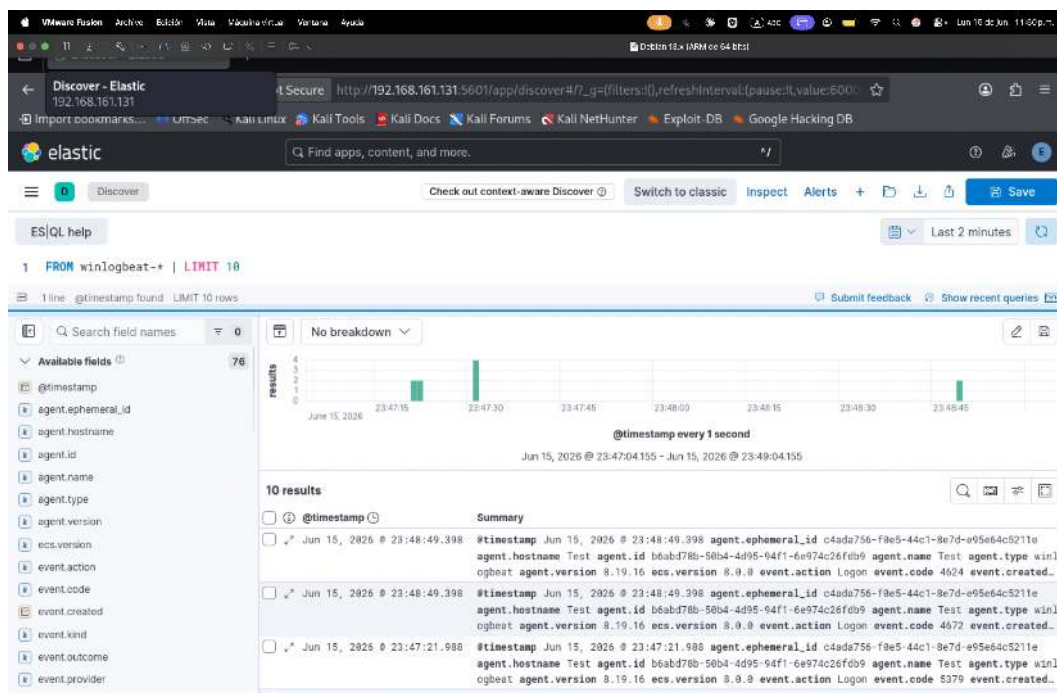


Figure 44: Estado RUNNING del servicio Winlogbeat.

5.4 Activación de la auditoría en Windows

No todos los eventos que Winlogbeat puede leer se generan automáticamente en Windows. Algunos canales y eventos se registran **por defecto** y otros requieren **activación explícita** de la auditoría (vía `auditpol` o, en el caso de PowerShell, vía Registry/Group Policy). A continuación se documentan los dos tipos y se justifica cuáles se activaron manualmente durante el workshop.

Canales que vienen activos por defecto

Table 2: Eventos que Windows registra sin necesidad de `auditpol`

Canal	IDs típicos	Justificación
System	7036, 7040, 7045, 6005, 6006, 1074	El <i>Service Control Manager</i> y el kernel registran de forma nativa los cambios de estado de servicios, instalación de servicios, reinicios y apagados. No requieren <code>auditpol</code> .
Application	1000, 8198, 16384, etc. (según el proveedor)	Cada aplicación instalada escribe sus propios eventos (errores, warnings, SPP de activación, Defender, etc.) sin que el usuario tenga que activar nada.
Windows PowerShell	400, 600, 800	La sesión clásica de PowerShell registra inicio del host, providers y ejecución de pipelines automáticamente.

Como ejemplo, durante el workshop observamos eventos 7040 de cambios en el servicio BITS sin haber activado ninguna auditoría adicional, lo cual confirma que el canal **System** está operativo por defecto.

Eventos del canal Security que requieren `auditpol`

El canal **Security** es distinto: la mayoría de sus eventos **no se registran** a menos que la subcategoría de auditoría correspondiente esté habilitada. Por eso debemos activarlas explícitamente para que Winlogbeat pueda capturarlas y enviarlas a Elasticsearch.

```

1  :: Inicio de sesión exitoso/fallido (4624, 4625, 4634, 4647)
2  auditpol /set /subcategory:"Logon" /success:enable /failure:enable
3  auditpol /set /subcategory:"Logoff" /success:enable
4
5  :: Creacion y terminacion de procesos (4688, 4689)
6  auditpol /set /subcategory:"Process Creation" /success:enable
7  auditpol /set /subcategory:"Process Termination" /success:enable
8
9  :: Cambios administrativos locales (4720, 4724, 4726, 4738)
10 auditpol /set /subcategory:"User Account Management" /success:enable /failure:
    enable
11
12 :: Cambios de grupos locales (4732, 4733)
13 auditpol /set /subcategory:"Security Group Management" /success:enable
14
15 :: Cambios de politicas (4719)
16 auditpol /set /subcategory:"Audit Policy Change" /success:enable
17
18 :: Cambios en reglas del firewall (4946, 4947, 4948)
19 auditpol /set /subcategory:"MPSSVC Rule-Level Policy Change" /success:enable /
    failure:enable
20 auditpol /set /subcategory:"Filtering Platform Policy Change" /success:enable

```

Listing 1: Activación de subcategorías de auditoría usadas en el workshop

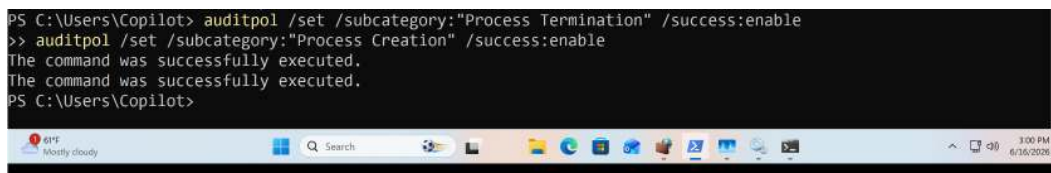


Figure 45: Activación de la auditoría *Process Termination*.

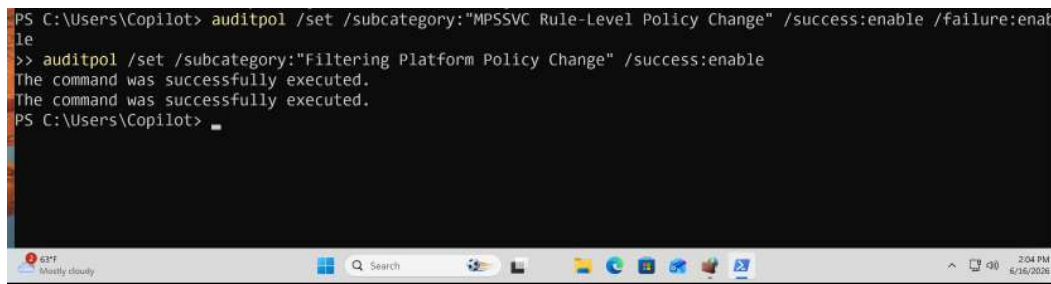


Figure 46: Activación de la auditoría *MPSSVC Rule-Level Policy Change* (cambios de firewall).

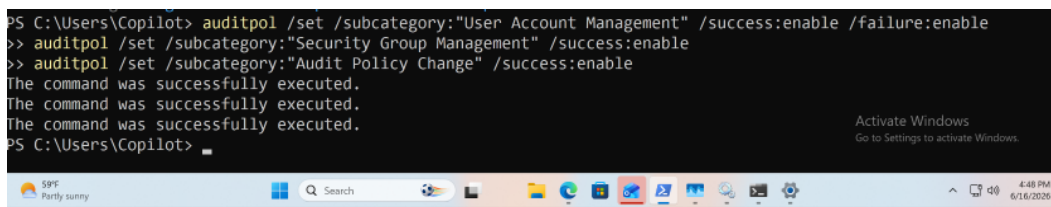


Figure 47: Activación de la auditoría *User Account Management*.

PowerShell Script Block Logging (4104) – vía Registry

El evento 4104 del canal Microsoft-Windows-PowerShell/Operational (que contiene el código exacto del *script block* ejecutado) **no se activa con auditpol**, sino mediante una clave del Registro

de Windows o una Group Policy:

```
1 $basePath = 'HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\  
   ScriptBlockLogging'  
2 if (-not (Test-Path $basePath)) {  
3     New-Item -Path $basePath -Force | Out-Null  
4 }  
5 Set-ItemProperty -Path $basePath -Name 'EnableScriptBlockLogging' -Value 1 -Type  
   DWord
```

Listing 2: Activación de Script Block Logging vía Registry

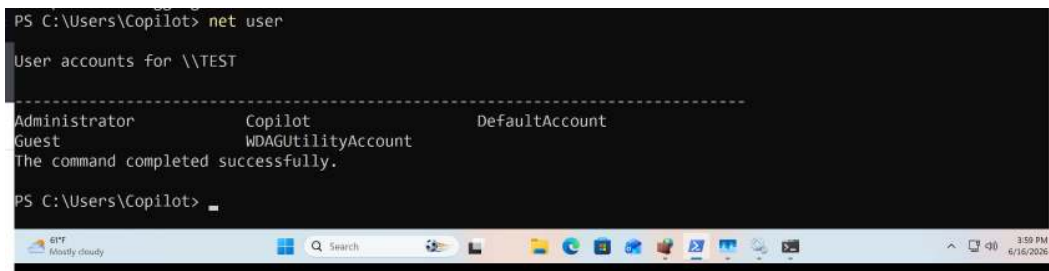


Figure 48: Activación de *PowerShell Script Block Logging* en el Registry.

Resumen

- **Canales System, Application y Windows PowerShell clásico:** vienen activos de fábrica, no se configuró nada extra.
- **Canal Security:** se activaron las subcategorías de Logon, Process Creation/Termination, User Account Management, Security Group Management, Audit Policy Change y MPSSVC Rule-Level Policy Change.
- **Canal Microsoft-Windows-PowerShell/Operational:** se habilitó Script Block Logging vía Registry para obtener el contenido literal del 4104.

5.5 Confirmación en Kibana de que los eventos llegan

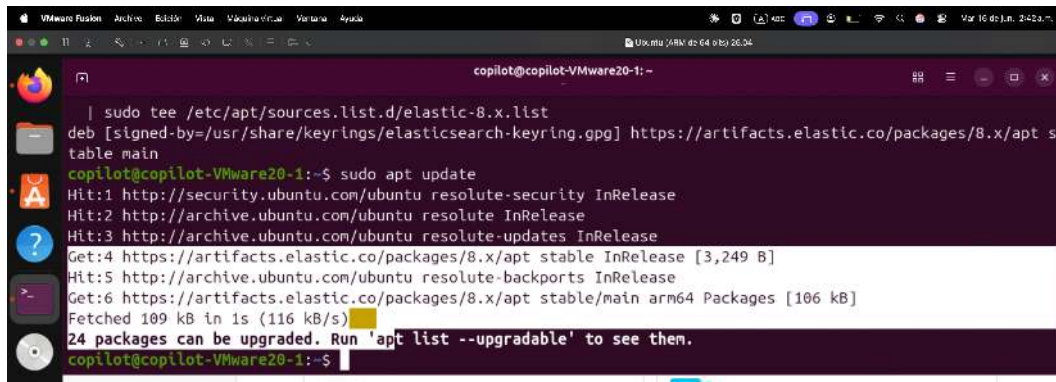
Una vez Winlogbeat está corriendo como servicio, la auditoría está activada y hay conexión con Elasticsearch, verificamos en Kibana (Discover, data view **winlogbeat-***) que los eventos del Event Viewer de Windows están siendo indexados correctamente. La **inspección detallada de los eventos** (con sus respectivos JSON y los campos relevantes identificados) se documenta en la **Fase 5: Exploración de eventos en Kibana**, y las **acciones controladas** que disparan cada tipo de evento se documentan en la **Fase 6: Generación controlada de eventos**.

6 Fase 4: Integración de métricas Linux con Metricbeat

Procedemos a descargar e instalar Metricbeat en la máquina Ubuntu con el siguiente comando:

```
1 sudo apt install metricbeat
```

Para esto primero repetimos el proceso de agregar el repositorio de Elasticsearch para poder instalar Metricbeat, pero ahora en la máquina Ubuntu. De este modo queda configurado el repositorio de Elastic en la máquina Ubuntu:



```
copilot@copilot-VMware20-1: ~  
| sudo tee /etc/apt/sources.list.d/elastic-8.x.list  
deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/8.x/apt stable main  
copilot@copilot-VMware20-1:~$ sudo apt update  
Hit:1 http://security.ubuntu.com/ubuntu resolute-security InRelease  
Hit:2 http://archive.ubuntu.com/ubuntu resolute InRelease  
Hit:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease  
Get:4 https://artifacts.elastic.co/packages/8.x/apt stable InRelease [3,249 B]  
Hit:5 http://archive.ubuntu.com/ubuntu resolute-backports InRelease  
Get:6 https://artifacts.elastic.co/packages/8.x/apt stable/main arm64 Packages [106 kB]  
Fetched 109 kB in 1s (116 kB/s)  
24 packages can be upgraded. Run 'apt list --upgradable' to see them.  
copilot@copilot-VMware20-1:~$
```

Figure 49: Configuración del repositorio Elastic 8.x en Ubuntu.

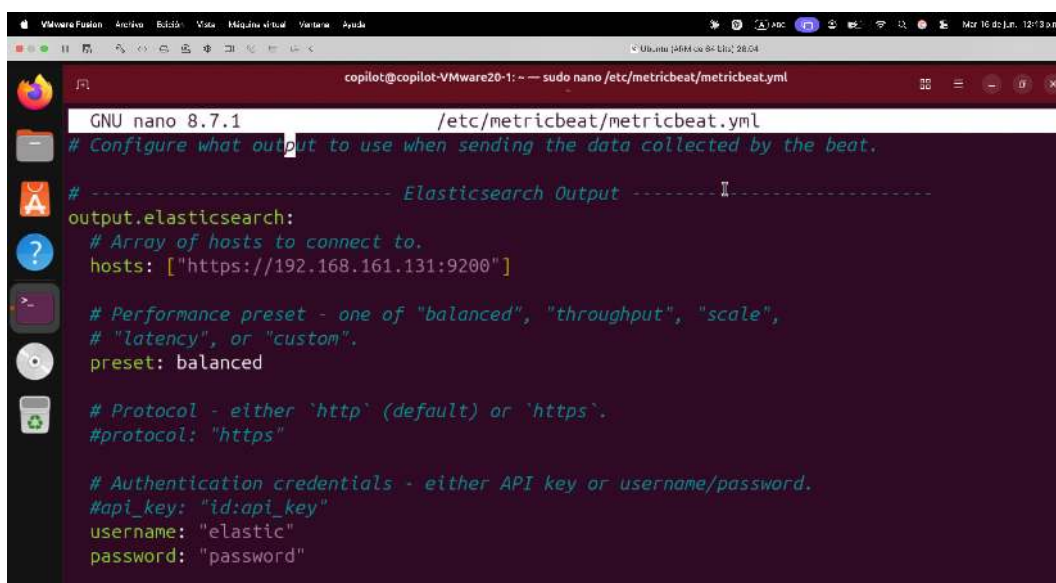
Por lo que ya podemos instalar Metricbeat con el siguiente comando:

```
1 sudo apt install metricbeat
```

6.1 Configuración de la salida hacia Elasticsearch

Ahora configuramos la salida de Metricbeat hacia Elasticsearch, editando el archivo de configuración:

```
1 sudo nano /etc/metricbeat/metricbeat.yml
```



```
copilot@copilot-VMware20-1: ~ — sudo nano /etc/metricbeat/metricbeat.yml  
GNU nano 8.7.1 /etc/metricbeat/metricbeat.yml  
# Configure what output to use when sending the data collected by the beat.  
# ----- Elasticsearch Output -----  
output.elasticsearch:  
# Array of hosts to connect to.  
hosts: ["https://192.168.161.131:9200"]  
# Performance preset - one of "balanced", "throughput", "scale",  
# "latency", or "custom".  
preset: balanced  
# Protocol - either 'http' (default) or 'https'.  
#protocol: "https"  
# Authentication credentials - either API key or username/password.  
#api_key: "id:api_key"  
username: "elastic"  
password: "password"
```

Figure 50: Configuración de output.elasticsearch con credenciales y ssl.verification_mode: none.

Y configuramos además la plantilla asociada a Kibana para Metricbeat:

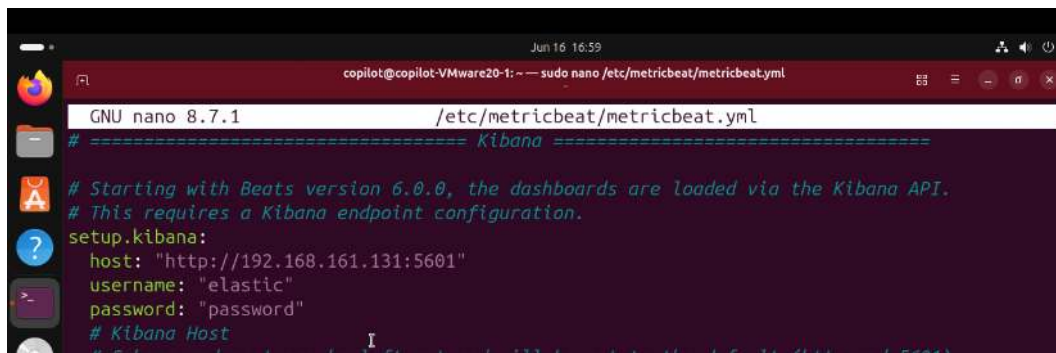


Figure 51: Configuración de setup.kibana en metricbeat.yml.

6.2 Habilitación del módulo system

Observamos en la siguiente imagen que en Metricbeat inicialmente casi todas las métricas están desactivadas, pero el módulo que necesitamos (**system**) ya se encuentra activo:



Figure 52: Listado de módulos: **system** habilitado.

Por lo que procedemos, dentro de las métricas del sistema, a activar las métricas de **CPU**, **memoria**, **red**, **disco**, **procesos** y **estado general del SO**.

6.3 Validación de la configuración

Validamos ahora la configuración de Metricbeat y verificamos que está funcionando correctamente:

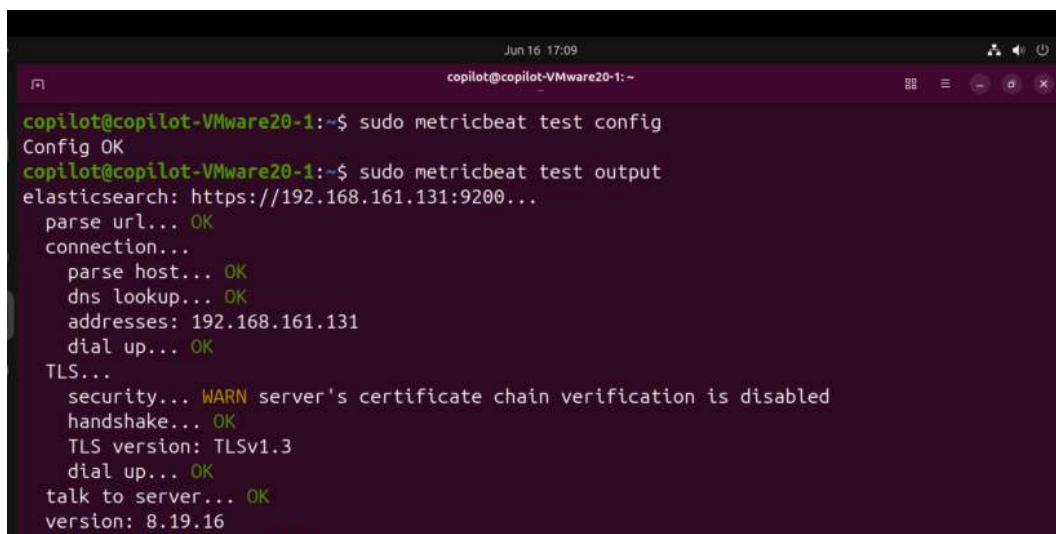


Figure 53: metricbeat test config y metricbeat test output con todos los OK.

6.4 Inicio del servicio

Configuramos en `systemd` y habilitamos Metricbeat, revisando el estado del servicio para verificar que se está ejecutando correctamente:

```
copilot@copilot-VMware20-1:~$ sudo systemctl daemon-reload
copilot@copilot-VMware20-1:~$ sudo systemctl enable metricbeat --now
Created symlink '/etc/systemd/system/multi-user.target.wants/metricbeat.service' → '/usr/lib/systemd/system/metricbeat.service'.
copilot@copilot-VMware20-1:~$ sudo systemctl status metricbeat
● metricbeat.service - Metricbeat sends metrics to Elasticsearch.
   Loaded: loaded (/usr/lib/systemd/system/metricbeat.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-06-16 17:28:30 -05; 2min 19s ago
 Invocation: b6852d31cada464d8ab8409d7d23f5a0
    Docs: https://www.elastic.co/beats/metricbeat
   Main PID: 7893 (metricbeat)
     Tasks: 8 (limit: 4484)
    Memory: 69.3M (peak: 72.3M)
       CPU: 3.589s
    CGroup: /system.slice/metricbeat.service
            └─7893 /usr/share/metricbeat/bin/metricbeat --environment systemd -c /etc/metr
```

Figure 54: Servicio metricbeat active (running).

6.5 Verificación en Kibana de la llegada de métricas

Verificamos el registro de eventos de Metricbeat en Kibana:

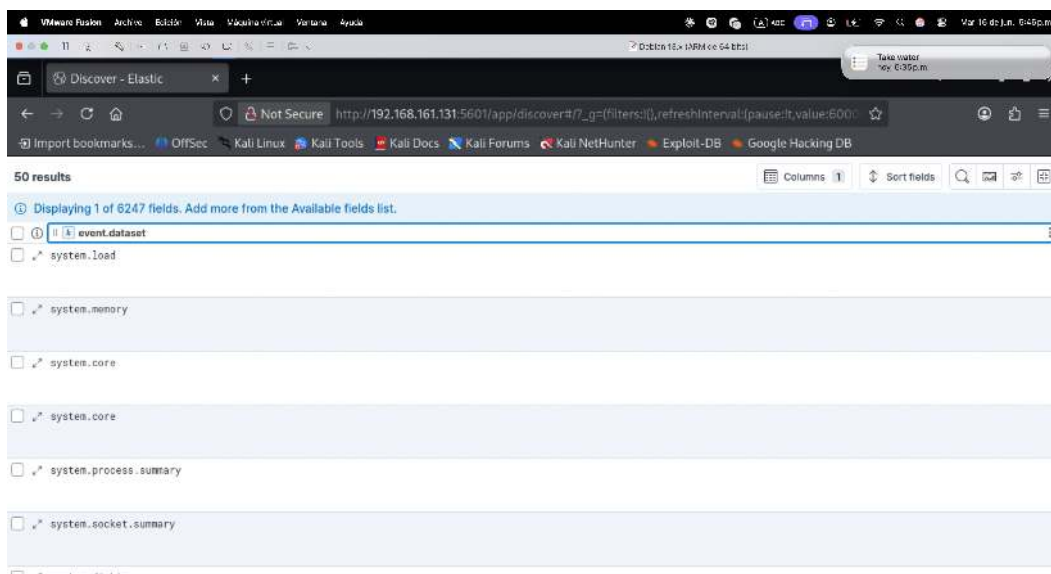


Figure 55: Discover mostrando métricas llegando desde Metricbeat al índice `metricbeat-*`.

Para mostrar únicamente los eventos de Metricbeat, modificamos la consulta a ES|QL:

```
1 FROM metricbeat-* | LIMIT 50
```

7 Fase 5: Exploración de eventos en Kibana

7.1 Acceso a Discover

Para navegar a **Discover** seleccionamos la opción **Analytics** en el menú lateral de Kibana y luego seleccionamos la opción **Discover**:

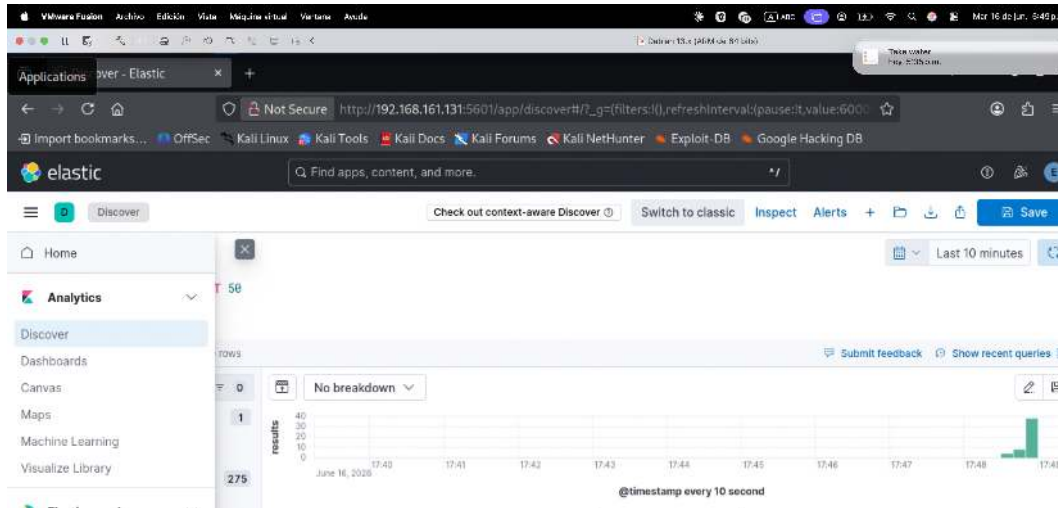


Figure 56: Acceso a la sección Discover desde el menú lateral de Kibana.

7.2 Filtrar eventos por host

Realizamos el filtro por `host.name` para el equipo Windows, el cual fue etiquetado como `Test`:

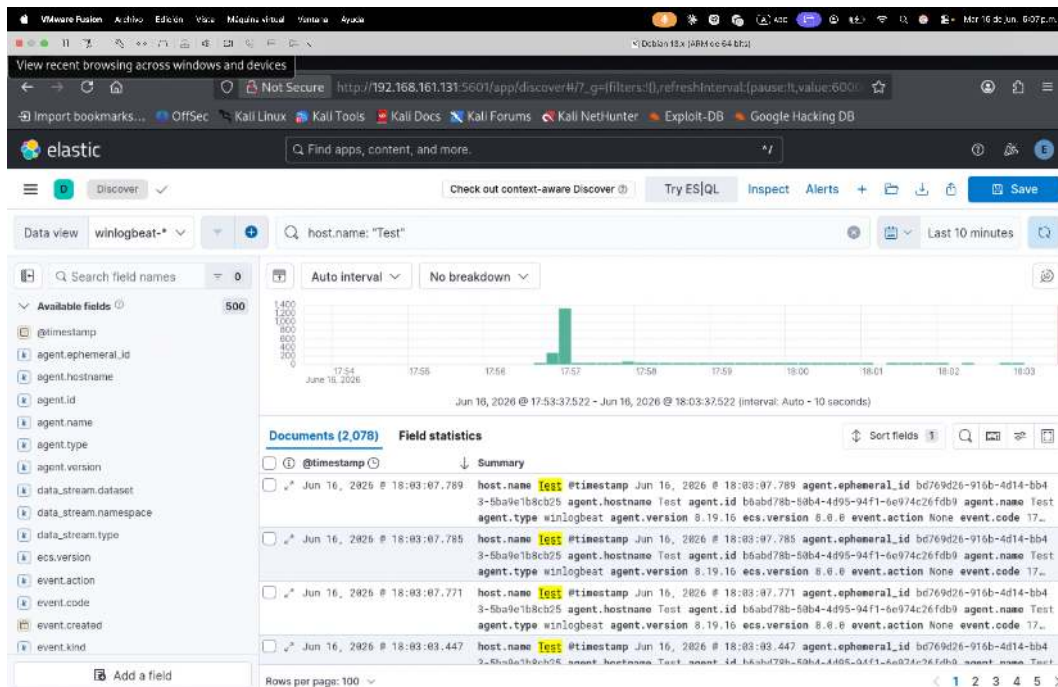


Figure 57: Filtro `host.name` : "Test" aplicado en Discover.

7.3 Filtrar eventos por agente

Realizamos el filtro por `agent.type` para mostrar solo los eventos de Winlogbeat:

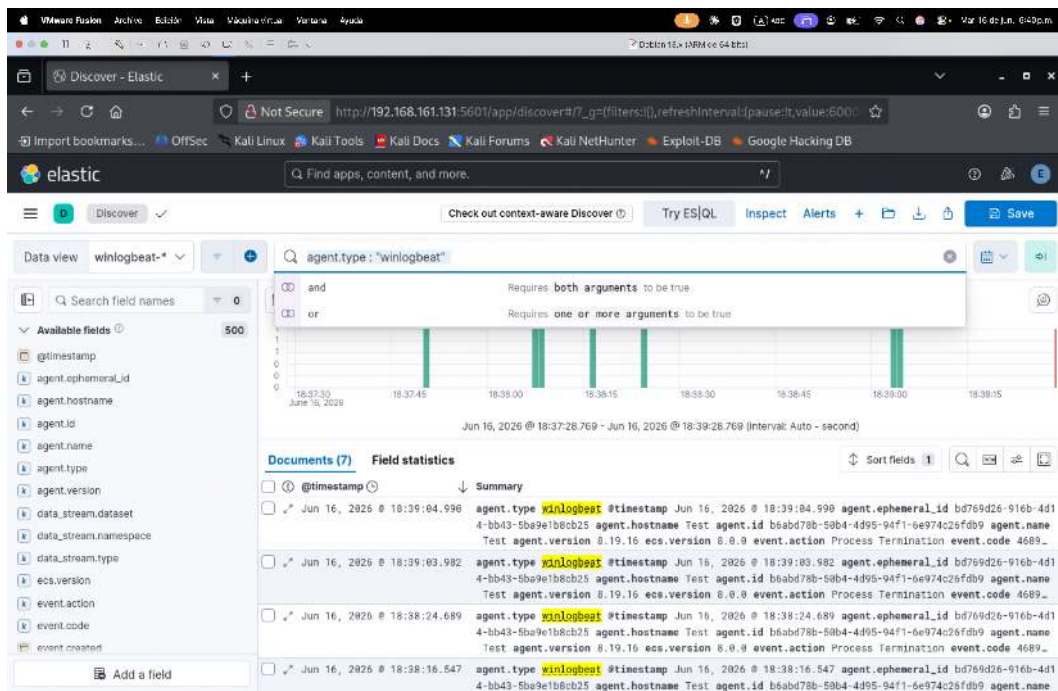


Figure 58: Filtro `agent.type` : "winlogbeat" aplicado en Discover.

Además, podemos aplicar filtros de agente por id, name, type, version, etc.:

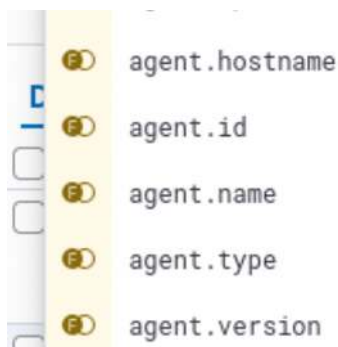


Figure 59: Campos disponibles del objeto `agent` para filtrar.

7.4 Filtrar eventos por tipo de evento

Cuando realizamos el filtro por `event.*`, tenemos las siguientes opciones: `action`, `id`, `duration`, `end`, `kind`, etc.:

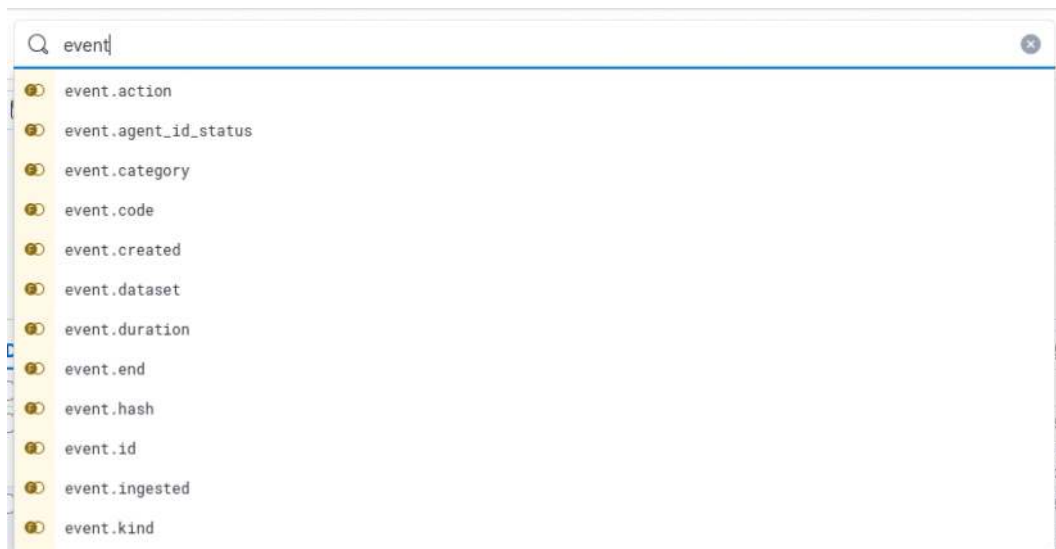


Figure 60: Campos disponibles del objeto `event` para filtrar.

7.5 Identificación de campos relevantes

En la parte anterior realizamos el filtro sobre todo por `event.code`, cuyos códigos identifican si es un evento a nivel de aplicación, de sistema, de servicios, de seguridad o de login fallido/exitoso. Los campos más relevantes identificados durante la exploración son:

- `host.name`: identifica la máquina que generó el evento (Test para Windows, copilot-VMware20-1 para Ubuntu).
- `agent.name` y `agent.type`: identifican el agente y su tipo.
- `event.code`: ID del evento (4624, 4625, 4948, etc.).
- `event.action`: acción asociada (Logon, Process Termination, Audit Policy Change, etc.).
- `event.dataset`: tipo de dato/fuente (`system.cpu`, `system.memory`, etc.).
- `user.name`, `winlog.event_data.TargetUserName`: usuario afectado.
- `source.ip`, `destination.ip`: direcciones IP involucradas (cuando aplica).

7.6 Métricas de Metricbeat – filtrado por `event.dataset`

En Metricbeat (configurado en Linux) podemos realizar el registro de las características del sistema configuradas, de la siguiente forma: `event.dataset: *`



Figure 61: Listado de `event.dataset` disponibles en `metricbeat-*`.

Obtenemos los registros de **CPU** con el filtro `event.dataset: system.cpu`, en los que podemos observar el porcentaje de uso de CPU, el número de procesos y el número de procesos en estado bloqueado:

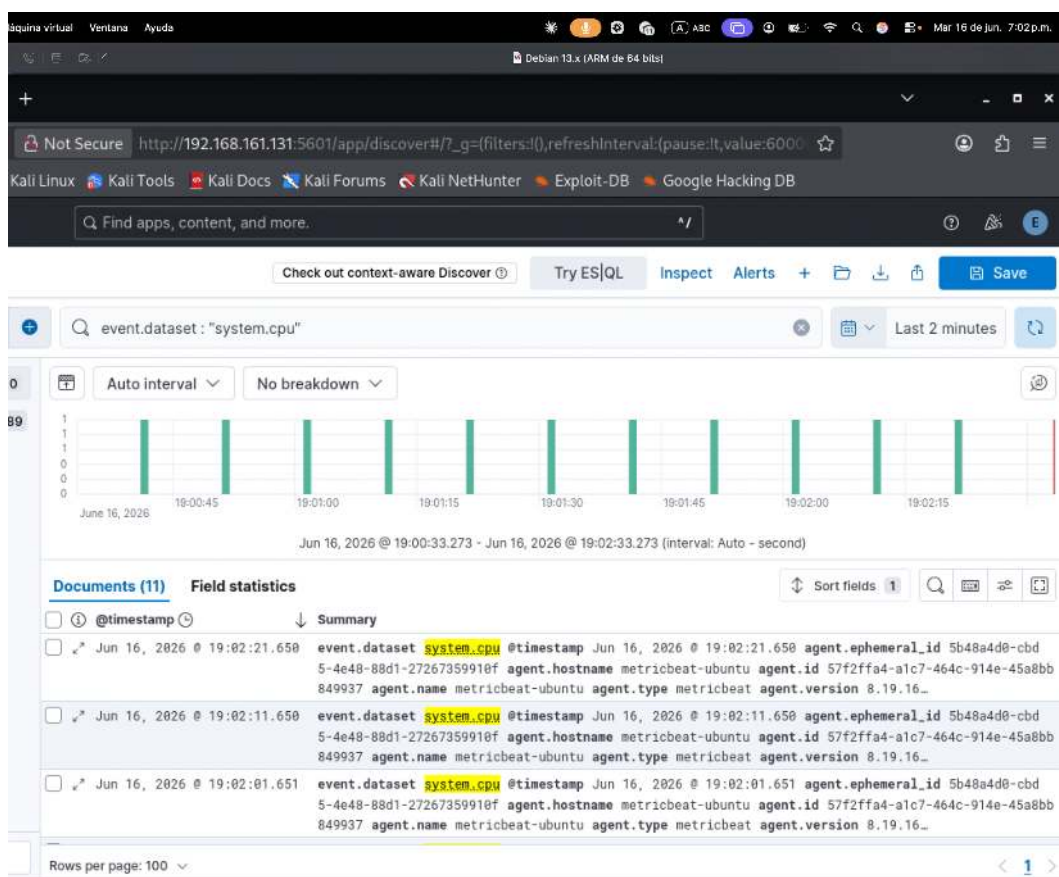


Figure 62: Filtro `event.dataset : "system.cpu"` mostrando métricas de CPU.

Obtenemos los registros de **memoria** con el filtro `event.dataset: system.memory`, en los que podemos observar el porcentaje de uso de memoria, entre otros:

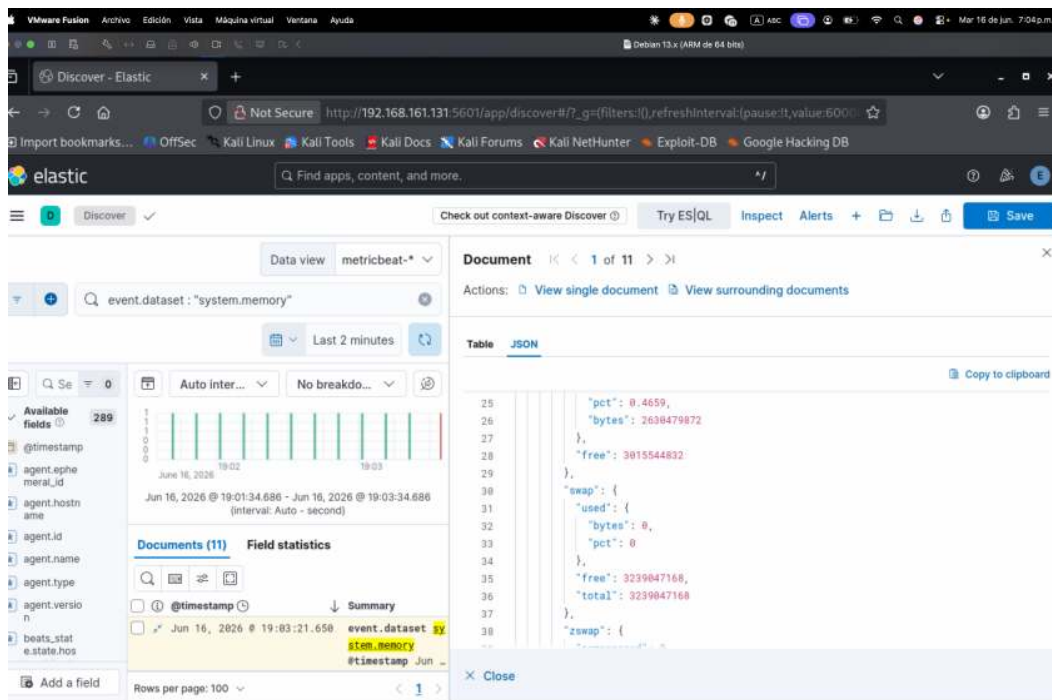


Figure 63: Filtro `event.dataset : "system.memory"` mostrando métricas de memoria.

7.7 Comparación logs Windows vs métricas Linux

En la siguiente imagen se muestra una comparación entre los eventos registrados en Windows con Winlogbeat y las métricas registradas en Linux con Metricbeat. Metricbeat registró en el mismo tiempo y con un uso similar de las máquinas Windows y Linux una cantidad de métricas en Linux (`copilot-VMware20-1`) mucho mayor que la cantidad de eventos en Windows (`Test`). Esto se debe a que Metricbeat registra una gran cantidad de métricas de sistema, mientras que Winlogbeat solo registra eventos relacionados con el sistema operativo y aplicaciones.



Figure 64: Comparación de volumen entre eventos de Windows (Winlogbeat) y métricas de Linux (Metricbeat) en Kibana.

Una vez ambos agentes están enviando datos al mismo Elasticsearch, podemos observar diferencias estructurales importantes entre lo que recolecta cada uno. La siguiente tabla resume las diferencias observadas durante la exploración en Discover:

Table 3: Comparación entre los datos enviados por Winlogbeat y Metricbeat

Aspecto	Winlogbeat (Windows)	Metricbeat (Ubuntu)
Tipo de dato	Eventos discretos (logs del Event Viewer).	Métricas numéricas continuas.
Naturaleza	Reactiva: se genera <i>cuando algo ocurre</i> (login, cambio de regla, ejecución de proceso).	Proactiva: se muestrea <i>cada N segundos</i> sin importar si hay actividad.
Frecuencia	Esporádica: depende de la actividad del sistema y del usuario.	Constante: <code>period: 10s</code> para CPU/memoria/red/procesos, <code>1m</code> para filesystem, <code>15m</code> para uptime.
Volumen relativo	Menor: cientos de documentos por hora en condiciones normales.	Mayor: miles de documentos por hora – aproximadamente un orden de magnitud más.
Índice destino	<code>winlogbeat-*</code>	<code>metricbeat-*</code>
Campo discriminante	<code>winlog.event_id</code> , <code>winlog.channel</code>	<code>event.dataset</code> (<code>system.cpu</code> , <code>system.memory</code> , etc.)
Útil para	Auditoría, forense, detección de intrusiones, cambios administrativos.	Monitoreo de salud, capacity planning, detección de degradación o saturación.
Ejemplos observados	4624 (login exitoso), 4625 (login fallido), 4948 (regla de firewall eliminada), 4720 (cuenta creada), 7040 (cambio de tipo de inicio de servicio), 4104 (script PowerShell).	<code>system.cpu</code> (uso de CPU), <code>system.memory</code> (uso de memoria), <code>system.network</code> , <code>system.process</code> , <code>system.filesystem</code> .

Consulta combinada en ES|QL

Las consultas combinadas en ES|QL permiten observar ambas fuentes (Winlogbeat y Metricbeat) en una misma vista, evidenciando la utilidad de centralizar logs y métricas en un único SIEM:

```

1 FROM winlogbeat-*, metricbeat-*
2 | WHERE @timestamp > NOW() - 30 minutes
3 | STATS count = COUNT(*) BY host.name, agent.type
4 | SORT count DESC

```

El resultado típico muestra una tabla en la que la fila de `copilot-VMware20-1 / metricbeat` tiene un *count* muy superior al de `Test / winlogbeat`. Esto refleja la diferencia estructural ya documentada en la tabla: las *métricas continuas* producen muchos más documentos que los *eventos discretos*, aunque ambos comparten el mismo ciclo de vida dentro del SIEM (ingesta, indexación, búsqueda y visualización).

Conclusión del análisis

La combinación de Winlogbeat y Metricbeat en una sola plataforma cubre los dos pilares fundamentales de un SOC:

- **Visibilidad de eventos de seguridad** (Winlogbeat): permite responder a la pregunta “¿qué pasó?” – quién se logueó, qué cambió, qué se ejecutó.
- **Visibilidad del estado del sistema** (Metricbeat): permite responder a “¿cómo está?” – saturación de CPU, memoria, disco o red.

La correlación de ambos en una misma línea de tiempo (mediante consultas sobre los dos índices) permite, por ejemplo, vincular un pico de CPU en Linux con un evento de PowerShell en Windows

ocurrido en el mismo intervalo, algo que sería muy difícil de detectar revisando los sistemas por separado. Esta es la justificación práctica de centralizar logs y métricas en un único SIEM.

El detalle de los documentos JSON observados en Discover para cada categoría de eventos (Security, System, Application, PowerShell, logins, servicios y cambios administrativos) se documenta en la **Fase 6: Generación controlada de eventos**, donde cada acción generadora aparece junto a su evento resultante.

8 Fase 6: Generación controlada de eventos

A continuación se documentan las **acciones ejecutadas** para generar de forma controlada cada categoría de evento sugerida por el workshop, junto con el documento JSON resultante observado en Kibana. La sección se divide en eventos controlados en Windows (vía Winlogbeat) y eventos controlados en Linux (vía Metricbeat). La activación previa de auditoría en Windows se documenta en la Sección 5.4 (Fase 3).

8.1 Eventos controlados en Windows

8.1.1 Eventos del sistema – Process Termination (4689)

Acción generadora: ejecutar cualquier programa que después termine. En el workshop observamos la terminación natural del proceso `RuntimeBroker.exe`. Con la auditoría *Process Termination* activada (ver Fase 3), Windows registra el evento 4689 en el canal Security sin acción adicional.

JSON resultante:

```
{
  "@timestamp": "2026-06-16T19:59:18.197Z",
  "host.name": "Test",
  "agent.name": "Test",
  "event.code": "4689",
  "event.action": "Process Termination",
  "event.outcome": "success",
  "winlog.channel": "Security",
  "winlog.event_data.ProcessName": "C:\\Windows\\System32\\RuntimeBroker.exe",
  "winlog.event_data.SubjectUserName": "Copilot",
  "message": "A process has exited. Process Name: RuntimeBroker.exe"
}
```

Listing 3: Evento 4689 – Process exited (RuntimeBroker.exe)

8.1.2 Eventos de seguridad – Firewall Rule Change (4948)

Acción generadora: creamos una nueva regla de firewall para permitir tráfico por el puerto 8888, y luego la eliminamos:



Figure 65: Creación de la regla ReglaDePruebaParaLogKibana en Windows Firewall.

Equivalente por línea de comandos:

```
1 netsh advfirewall firewall add rule name="ReglaDePruebaParaLogKibana" '
2   dir=in action=allow protocol=TCP localport=8888
3 netsh advfirewall firewall delete rule name="ReglaDePruebaParaLogKibana"
```

JSON resultante:

```
{
  "@timestamp": "2026-06-16T19:00:17.737Z",
  "host.name": "Test",
  "agent.name": "Test",
  "event.code": "4948",
  "event.outcome": "success",
  "winlog.channel": "Security",
  "winlog.event_data.RuleName": "ReglaDePruebaParaLogKibana",
  "winlog.event_data.ProfileChanged": "All",
  "message": "A rule was deleted from the Windows Firewall exception list."
}
```

Listing 4: Evento 4948 – Firewall rule deleted

8.1.3 Evento de aplicación – custom (999)

Acción generadora: ejecución del cmdlet Write-EventLog en PowerShell (admin), creando previamente el *source* WorkshopSIEM en el canal Application:

```
1 if (-not [System.Diagnostics.EventLog]::SourceExists("WorkshopSIEM")) {
2     [System.Diagnostics.EventLog]::CreateEventSource("WorkshopSIEM", "Application"
3 }
4 Write-EventLog -LogName Application -Source "WorkshopSIEM" '
5 -EntryType Information -EventId 999 '
6 -Message "Segundo evento de prueba del workshop"
```

JSON resultante:

```
{
  "@timestamp": "2026-06-16T21:36:35.128Z",
  "host.name": "Test",
  "agent.name": "Test",
  "event.code": "999",
  "event.provider": "WorkshopSIEM",
  "winlog.channel": "Application",
  "log.level": "information",
  "message": "Segundo evento de prueba del workshop"
}
```

Listing 5: Evento 999 – Application custom (WorkshopSIEM)

8.1.4 Eventos de PowerShell – Script Block Logging (4104)

Acción generadora: con *Script Block Logging* habilitado (ver Fase 3), basta con abrir una sesión de PowerShell y ejecutar cualquier cmdlet. En el workshop se usó:

```
1 Get-ItemProperty -Path 'HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\
   ScriptBlockLogging' '
2 -Name EnableScriptBlockLogging -ErrorAction SilentlyContinue
```

JSON resultante:

```
{
  "@timestamp": "2026-06-16T20:57:32.518Z",
  "host.name": "Test",
  "agent.name": "Test",
  "event.code": "4104",
  "event.action": "Execute a Remote Command",
  "winlog.channel": "Microsoft-Windows-PowerShell/Operational",
  "winlog.user.name": "Copilot",
  "winlog.event_data.ScriptBlockText": "Get-ItemProperty -Path 'HKLM:\\SOFTWARE\\Policies\\Microsoft\\Windows
    \\PowerShell\\ScriptBlockLogging' -Name EnableScriptBlockLogging"
}
```

Listing 6: Evento 4104 – PowerShell Script Block

8.1.5 Eventos de login – fallido (4625) y exitoso (4624)

Login fallido (4625). Se intentó acceder con contraseña incorrecta a la cuenta Copilot, lo que dispara el evento 4625 en Security (LogonType: 2 = interactivo, FailureReason: *Unknown user name or bad password*).

```
1 :: Equivalente
2 runas /user:Copilot cmd
```

JSON resultante:

```
{
  "@timestamp": "2026-06-16T18:19:49.179Z",
  "host.name": "Test",
```

```

"agent.name": "Test",
"event.code": "4625",
"event.action": "Logon",
"event.outcome": "failure",
"winlog.channel": "Security",
"winlog.event_data.TargetUserName": "Copilot",
"winlog.event_data.LogonType": "2",
"winlog.event_data.IpAddress": "127.0.0.1",
"winlog.event_data.FailureReason": "Unknown user name or bad password",
"message": "An account failed to log on."
}

```

Listing 7: Evento 4625 – Logon failure

Login exitoso (4624). Cualquier autenticación exitosa (interactiva, de servicio o de red) genera el evento 4624. En la muestra capturada, el subject es **SYSTEM** con **LogonType: 5** (servicio), lo que confirma que Windows genera muchos 4624 incluso sin intervención del usuario.

JSON resultante:

```

{
  "@timestamp": "2026-06-16T18:22:30.259Z",
  "host.name": "Test",
  "agent.name": "Test",
  "event.code": "4624",
  "event.action": "Logon",
  "event.outcome": "success",
  "winlog.channel": "Security",
  "winlog.event_data.TargetUserName": "SYSTEM",
  "winlog.event_data.LogonType": "5",
  "winlog.event_data.ProcessName": "C:\\Windows\\System32\\services.exe",
  "message": "An account was successfully logged on (Logon Type 5 = service).",
}

```

Listing 8: Evento 4624 – Logon success

8.1.6 Eventos relacionados con servicios – Service Configuration (7040)

Acción generadora: cambiar el tipo de inicio de un servicio. Se observó un evento 7040 natural cuando el servicio BITS (*Background Intelligent Transfer Service*) cambió de **auto start** a **demand start**. Este canal (**System**) no requiere **auditpol** y se registra automáticamente.

Equivalente manual:

```

1 sc config Spooler start= demand
2 sc config Spooler start= auto

```

JSON resultante:

```

{
  "@timestamp": "2026-06-16T21:38:21.997Z",
  "host.name": "Test",
  "agent.name": "Test",
  "event.code": "7040",
  "event.provider": "Service Control Manager",
  "winlog.channel": "System",
  "winlog.event_data.param1": "Background Intelligent Transfer Service",
  "winlog.event_data.param2": "auto start",
  "winlog.event_data.param3": "demand start",
  "message": "The start type of BITS was changed from auto start to demand start."
}

```

Listing 9: Evento 7040 – Service start type changed (BITS)

8.1.7 Eventos de cambios administrativos locales – User Account Created (4720)

Acción generadora: desde **compmgmt.msc** (Computer Management) → Local Users and Groups → Users, se creó un nuevo usuario llamado **PruebaUsuario**:

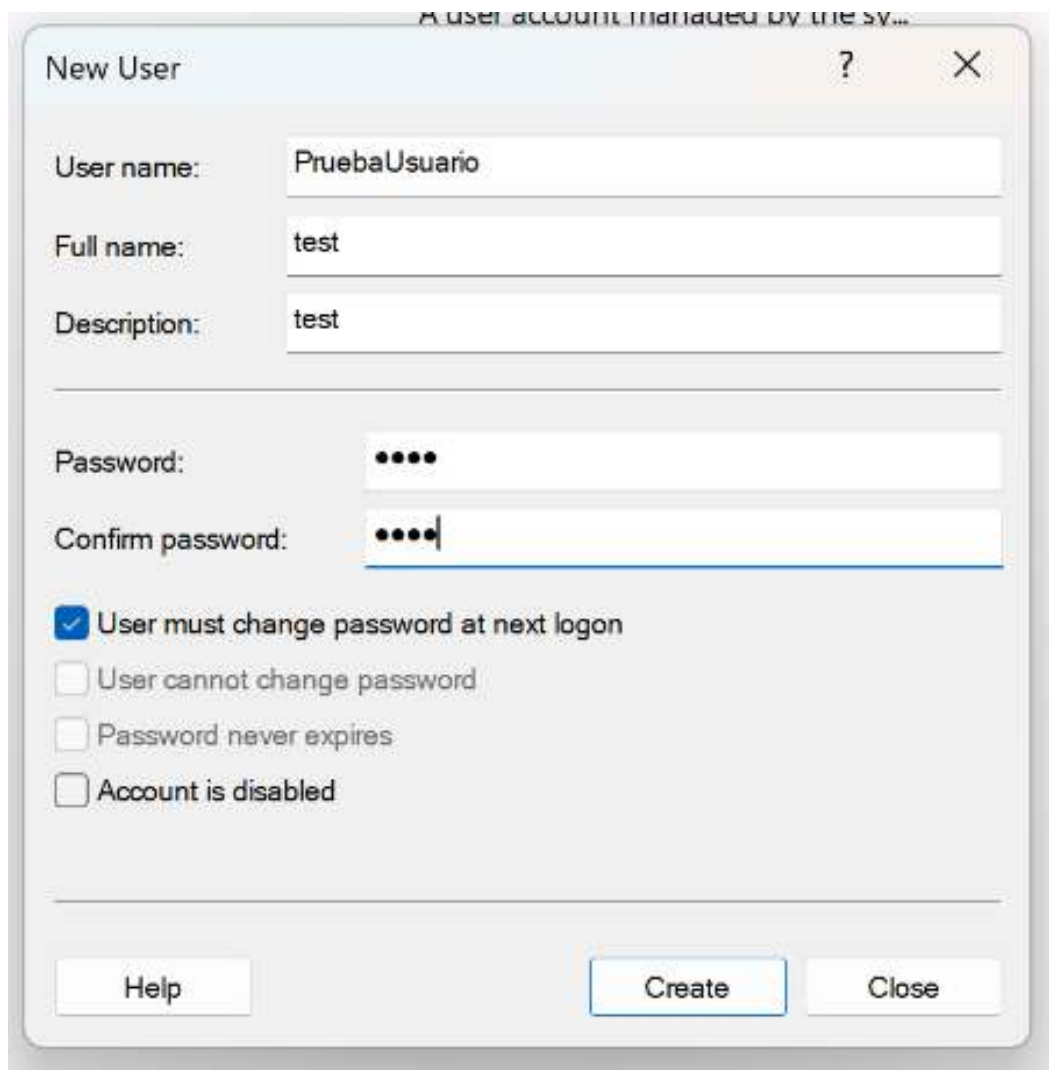


Figure 66: Creación del usuario PruebaUsuario desde Computer Management.

Equivalente por línea de comandos:

```
1 net user PruebaUsuario Workshop2026! /add
```

JSON resultante:

```
{
  "@timestamp": "2026-06-16T21:50:56.577Z",
  "host.name": "Test",
  "agent.name": "Test",
  "event.code": "4720",
  "event.outcome": "success",
  "winlog.channel": "Security",
  "winlog.event_data.TargetUserName": "PruebaUsuario",
  "winlog.event_data.SubjectUserName": "Copilot",
  "winlog.event_data.SamAccountName": "PruebaUsuario",
  "message": "A user account was created. New Account: PruebaUsuario. Subject: Copilot."
}
```

Listing 10: Evento 4720 – User account created

8.1.8 Resumen de eventos controlados en Windows

Table 4: Resumen de acciones controladas y eventos resultantes en Windows

ID	Canal	Acción ejecutada en Windows
4689	Security	Terminación natural de procesos con auditoría <i>Process Termination</i> activa.
4948	Security	Creación y eliminación de la regla <i>ReglaDePruebaParaLogKibana</i> en Windows Firewall (puerto TCP 8888).
999	Application	<i>Write-EventLog</i> con el source custom <i>WorkshopSIEM</i> .
4104	PowerShell/Operational	Ejecución del cmdlet <i>Get-ItemProperty</i> con <i>Script Block Logging</i> habilitado.
4625	Security	Intento de login con contraseña incorrecta (<i>LogonType: 2</i>).
4624	Security	Inicio de sesión exitoso de SYSTEM (<i>LogonType: 5</i>).
7040	System	Cambio del tipo de inicio del servicio BITS (<i>auto start</i> → <i>demand start</i>).
4720	Security	Creación del usuario local <i>PruebaUsuario</i> desde Computer Management.

8.2 Eventos controlados en Linux

Para generar eventos controlados en Linux, podemos utilizar el comando **stress** para generar carga en el sistema y observar cómo se registran las métricas en Kibana. Primero instalamos **stress**:

```
1 sudo apt install stress
```

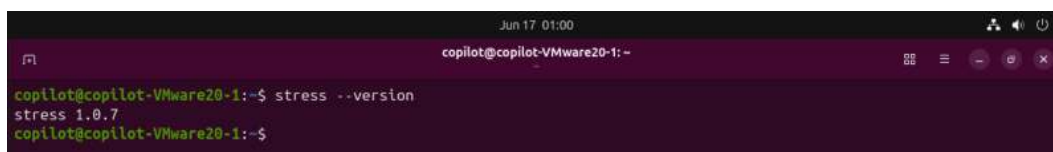


Figure 67: Instalación de **stress** en Ubuntu.

8.2.1 CPU

Luego ejecutamos el comando **stress** para generar carga en el sistema. Por ejemplo, para generar carga en la CPU podemos ejecutar:

```
1 stress --cpu 2 --timeout 60s
```



Figure 68: Ejecución de **stress -cpu 2 -timeout 60s** en Ubuntu.

Observamos que se generó carga en la CPU y se registraron las métricas en Kibana, donde observamos que el porcentaje de uso de CPU es del 100%. Este tipo de registros se pueden observar con el filtro `event.dataset: system.cpu`.

```
{
  "@timestamp": "2026-06-17T06:25:41.604Z",
```

```

"host.name": "copilot-vmware20-1",
"host.cpu.usage": 1,
"agent.name": "metricbeat-ubuntu",
"agent.type": "metricbeat",
"event.dataset": "system.cpu",
"event.module": "system",
"metricset.name": "cpu",
"metricset.period": 10000,
"system.cpu.cores": 2,
"system.cpu.total.pct": 2,
"system.cpu.total.norm.pct": 1,
"system.cpu.user.pct": 1.9732,
"system.cpu.user.norm.pct": 0.9866,
"system.cpu.system.pct": 0.0258,
"system.cpu.idle.pct": 0
}

```

Listing 11: Evento `system.cpu` durante la carga (CPU al 100%)

8.2.2 Memoria

Para generar carga en la memoria, primero verificamos el uso de memoria con:

```
1 free -h
```

Observamos que el uso de memoria es de 2.3 GiB y hay 1.6 GiB de memoria libre, por lo que podemos generar carga en la memoria de 1.0 GiB.

```

Jun 17 01:31
copilot@copilot-VMware20-1: ~
copilot@copilot-VMware20-1:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:           5.3Gi       2.3Gi       1.6Gi         35Mi       1.5Gi       2.9Gi
Swap:          3.0Gi           0B       3.0Gi
copilot@copilot-VMware20-1:~$

```

Figure 69: Verificación inicial de memoria con `free -h`.

Para esto ejecutamos el siguiente comando:

```
1 stress --vm 1 --vm-bytes 4000M --timeout 60
```

```

Jun 17 01:56
copilot@copilot-VMware20-1: ~
copilot@copilot-VMware20-1:~$ stress --vm 1 --vm-bytes 2000M --timeout 60
stress: info: [10227] dispatching hogs: 0 cpu, 0 io, 1 vm, 0 hdd
stress: info: [10227] successful run completed in 60s
copilot@copilot-VMware20-1:~$ stress --vm 1 --vm-bytes 3000M --timeout 60
stress: info: [10233] dispatching hogs: 0 cpu, 0 io, 1 vm, 0 hdd
stress: info: [10233] successful run completed in 60s
copilot@copilot-VMware20-1:~$

```

Figure 70: Ejecución de `stress -vm 1 -vm-bytes 4000M -timeout 60`.

Aplicamos el filtro `event.dataset: system.memory` para observar las métricas de memoria en Kibana, donde observamos que el porcentaje de uso de memoria es cercano al 100% debido a la carga que generamos:

```

{
"@timestamp": "2026-06-17T06:53:11.633Z",
"host.name": "copilot-vmware20-1",
"agent.name": "metricbeat-ubuntu",
"agent.type": "metricbeat",
"event.dataset": "system.memory",

```

```
"metricset.name": "memory",
"system.memory.total": 5646024704,
"system.memory.used.bytes": 5335044096,
"system.memory.used.pct": 0.9449,
"system.memory.actual.used.bytes": 5288751104,
"system.memory.actual.used.pct": 0.9367,
"system.memory.free": 310980608,
"system.memory.cached": 178028544,
"system.memory.swap.used.bytes": 165974016,
"system.memory.swap.used.pct": 0.0512
}
```

Listing 12: Evento `system.memory` durante la carga (uso $\approx 94\%$)

El valor `actual.used.pct = 0.9367` y la activación de `swap (0.0512)` confirman saturación real de memoria: el kernel comenzó a mover páginas a disco al agotarse la RAM física.

8.2.3 Red

Para generar carga en la red simulamos un ataque DoS desde la máquina Kali hacia la máquina Ubuntu. Para esto usamos el siguiente comando en la máquina Kali (192.168.161.131):

```
1 sudo ping -f -s 65500 192.168.161.133
```

Y aplicamos el filtro `event.dataset: system.network` para observar las métricas de red en Kibana, donde observamos un aumento en el número de paquetes transmitidos y recibidos debido a la carga generada:

```
{
"@timestamp": "2026-06-17T07:14:11.604Z",
"host.name": "copilot-vmware20-1",
"host.ip": ["192.168.161.133"],
"agent.name": "metricbeat-ubuntu",
"event.dataset": "system.network",
"metricset.name": "network",
"host.network.ingress.bytes": 151045125,
"host.network.ingress.packets": 101125,
"host.network.egress.bytes": 150650518,
"host.network.egress.packets": 101136
}
```

Listing 13: Evento `system.network` durante el ataque (alto tráfico)

Se observa un aumento sustancial en el número de paquetes transmitidos y recibidos debido a la carga generada en la red. Si se compara con un registro de red normal, se puede observar que el número de paquetes es mucho menor:

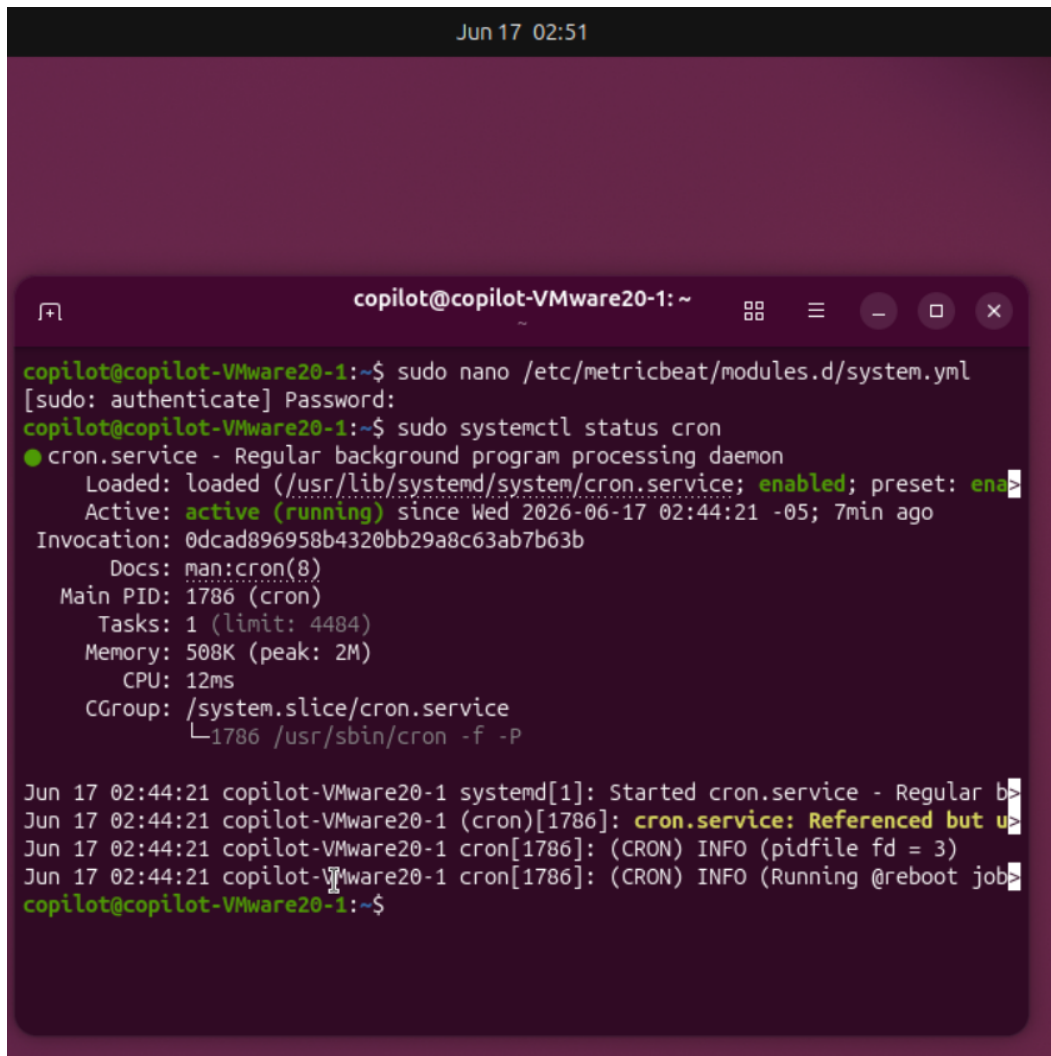
```
{
"@timestamp": "2026-06-17T06:51:01.613Z",
"host.name": "copilot-vmware20-1",
"agent.name": "metricbeat-ubuntu",
"event.dataset": "system.network",
"metricset.name": "network",
"system.network.name": "lo",
"system.network.in.bytes": 46121,
"system.network.in.packets": 559,
"system.network.out.bytes": 46121,
"system.network.out.packets": 559,
"system.network.in.errors": 0,
"system.network.in.dropped": 0
}
```

Listing 14: Evento `system.network` en condiciones normales (interfaz lo)

El contraste entre 101,125 paquetes/intervalo durante el ataque y 559 paquetes/intervalo en estado normal (un aumento de $\sim 180\times$) es la huella clara de un DoS detectado por el SIEM.

8.2.4 Inicio o detención de un servicio

Para esto usamos el servicio `cron`, que ya se encuentra instalado en la máquina Ubuntu. Verificamos que se encuentra activo:



```

Jun 17 02:51

copilot@copilot-VMware20-1: ~
copilot@copilot-VMware20-1:~$ sudo nano /etc/metricbeat/modules.d/system.yml
[sudo: authenticate] Password:
copilot@copilot-VMware20-1:~$ sudo systemctl status cron
● cron.service - Regular background program processing daemon
   Loaded: loaded (/usr/lib/systemd/system/cron.service; enabled; preset: ena>
   Active: active (running) since Wed 2026-06-17 02:44:21 -05; 7min ago
  Invocation: 0dcad896958b4320bb29a8c63ab7b63b
     Docs: man:cron(8)
    Main PID: 1786 (cron)
      Tasks: 1 (limit: 4484)
     Memory: 508K (peak: 2M)
        CPU: 12ms
    CGroup: /system.slice/cron.service
            └─1786 /usr/sbin/cron -f -P

Jun 17 02:44:21 copilot-VMware20-1 systemd[1]: Started cron.service - Regular b>
Jun 17 02:44:21 copilot-VMware20-1 (cron)[1786]: cron.service: Referenced but u>
Jun 17 02:44:21 copilot-VMware20-1 cron[1786]: (CRON) INFO (pidfile fd = 3)
Jun 17 02:44:21 copilot-VMware20-1 cron[1786]: (CRON) INFO (Running @reboot job>
copilot@copilot-VMware20-1:~$
  
```

Figure 71: Estado active (running) del servicio `cron` en Ubuntu.

Por lo que procedemos a detener el servicio `cron`:

```
1 sudo systemctl stop cron
```

Lo cual genera un evento de detención del servicio `cron`, observable aplicando el filtro `event.dataset: system.service` en Kibana. El estado del servicio cambió a `inactive` y el sub-estado a `dead`:

```

{
  "@timestamp": "2026-06-17T08:01:07.369Z",
  "host.name": "copilot-vmware20-1",
  "agent.name": "metricbeat-ubuntu",
  "event.dataset": "system.service",
  "metricset.name": "service",
  "system.service.name": "cron.service",
  "system.service.state": "inactive",
  "system.service.sub_state": "dead",
  "system.service.exec_code": "killed",
  "system.service.load_state": "loaded",
  "system.service.unit_file.state": "enabled",
  "system.service.state_since": "2026-06-17T07:56:13.541Z",
  "process.pid": 4966,
  "process.exit_code": 15,
}
  
```

```

"systemd.unit": "cron.service",
"systemd.fragment_path": "/usr/lib/systemd/system/cron.service"
}

```

Listing 15: Evento `system.service` – cron detenido

8.2.5 Revisión de procesos activos

Para esto usamos el comando `ps aux` para listar los procesos activos en la máquina Ubuntu, el cual nos muestra una lista de procesos con su PID, usuario, uso de CPU y memoria, entre otros datos:

```

copilot@copilot-VMware20-1:~$ ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root             1  1.0  0.3 27460 18180 ?        Ss   02:44    0:20 /usr/lib/syst
root             2  0.0  0.0      0     0 ?        S    02:44    0:00 [kthreadd]
root             3  0.0  0.0      0     0 ?        S    02:44    0:00 [pool_workque
root             4  0.0  0.0      0     0 ?        I<   02:44    0:00 [kworker/R-rc
root             5  0.0  0.0      0     0 ?        I<   02:44    0:00 [kworker/R-sy
root             6  0.0  0.0      0     0 ?        I<   02:44    0:00 [kworker/R-kv
root             7  0.0  0.0      0     0 ?        I<   02:44    0:00 [kworker/R-sl
root             8  0.0  0.0      0     0 ?        I<   02:44    0:00 [kworker/R-ne
root            10  0.0  0.0      0     0 ?        I<   02:44    0:00 [kworker/0:0H
root            13  0.0  0.0      0     0 ?        I<   02:44    0:00 [kworker/R-mm
root            14  0.0  0.0      0     0 ?        S    02:44    0:00 [ksoftirqd/0]
root            15  0.0  0.0      0     0 ?        I    02:44    0:00 [rcu_preempt]
root            16  0.0  0.0      0     0 ?        S    02:44    0:00 [rcu_exp_par_
root            17  0.0  0.0      0     0 ?        S    02:44    0:00 [rcu_exp_gp_k
root            18  0.0  0.0      0     0 ?        S    02:44    0:00 [migration/0]
root            19  0.0  0.0      0     0 ?        S    02:44    0:00 [idle_inject/
root            20  0.0  0.0      0     0 ?        S    02:44    0:00 [cpuhp/0]
root            21  0.0  0.0      0     0 ?        S    02:44    0:00 [cpuhp/1]
root            22  0.0  0.0      0     0 ?        S    02:44    0:00 [idle_inject/
root            23  0.0  0.0      0     0 ?        S    02:44    0:00 [idle_inject/

```

Figure 72: Listado de procesos activos en Ubuntu con `ps aux`.

Aplicamos el filtro `event.dataset: system.process` para observar las métricas de procesos en Kibana, donde observamos una lista de procesos con su PID, usuario, uso de CPU y memoria, entre otros datos:

```

{
  "@timestamp": "2026-06-17T08:19:26.895Z",
  "host.name": "copilot-vmware20-1",
  "agent.name": "metricbeat-ubuntu",
  "event.dataset": "system.process",
  "metricset.name": "process",
  "user.name": "copilot",
  "process.name": "mutter-x11-frames",
  "process.pid": 4215,
  "process.parent.pid": 3315,
  "process.executable": "/usr/libexec/mutter-x11-frames",
  "process.command_line": "/usr/libexec/mutter-x11-frames",
  "process.working_directory": "/home/copilot",
  "process.state": "sleeping",
  "process.cpu.start_time": "2026-06-17T07:45:31.000Z",
  "process.cpu.pct": 0,
  "process.memory.pct": 0.0287,
  "system.process.memory.rss.bytes": 162242560,
  "system.process.memory.size": 1177382912,
  "system.process.num_threads": 11,
  "system.process.fd.open": 14
}

```

Listing 16: Evento `system.process` – proceso `mutter-x11-frames`

Con esto se cubren las categorías exigidas por el workshop para Linux: aumento temporal de uso de CPU y memoria, actividad de red, control de servicios y revisión de procesos activos.

9 Fase 7: Creación de visualizaciones y dashboard (Bonus)

A continuación se documenta la construcción del dashboard en Kibana con las visualizaciones sugeridas por el workshop. Cada visualización fue creada mediante el editor **Lens** y agregada al mismo dashboard “*Workshop SIEM – BeTek*”.

9.1 Acceso al editor de Dashboards

En el menú lateral de Kibana, dentro de **Analytics**, se selecciona **Dashboard** y luego el botón **Create Dashboard**:

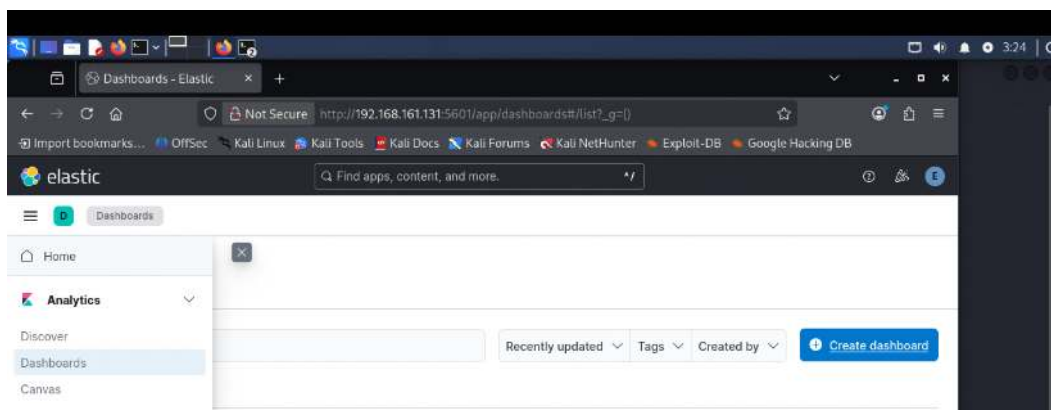


Figure 73: Acceso a la sección Dashboards y creación de un dashboard nuevo en Kibana.

A continuación se selecciona **Create new visualization** para acceder al editor Lens:

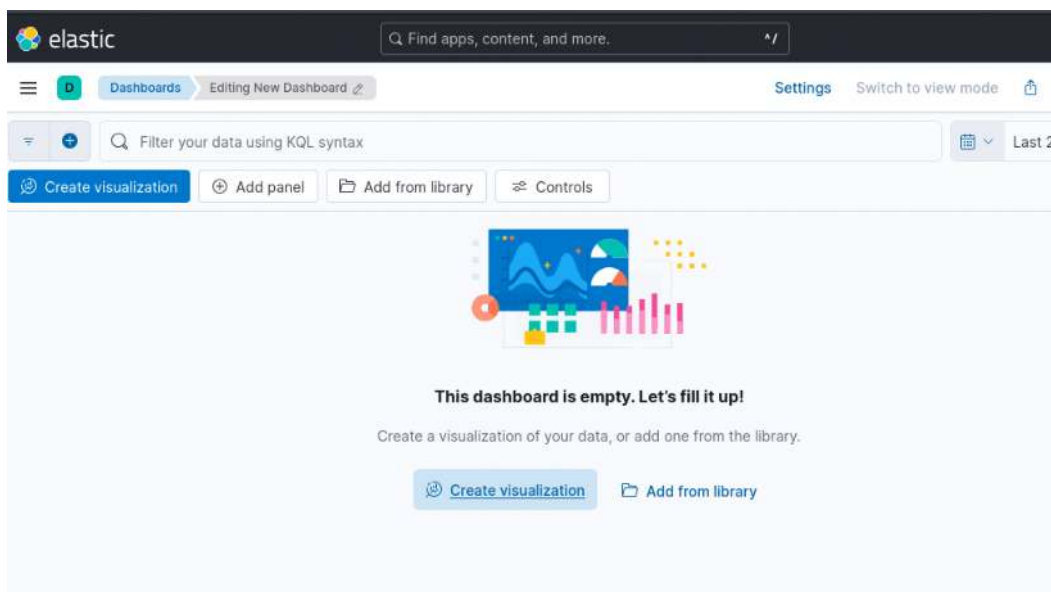


Figure 74: Botón *Create visualization* dentro del dashboard en edición.

9.2 Visualización 1 – Cantidad de eventos por host

Paso a paso en Lens:

1. Tipo de gráfica: **Bar vertical**.
2. Data view: `winlogbeat-*,metricbeat-*` (combinado).

3. Rango de tiempo: **Last 24 hours**.
4. **Horizontal axis**: arrastrar el campo `host.name`, función *Top values*, Rank by Count of records.
5. **Vertical axis**: arrastrar Records (Count of records).
6. Título del panel: *Cantidad de eventos por host*.

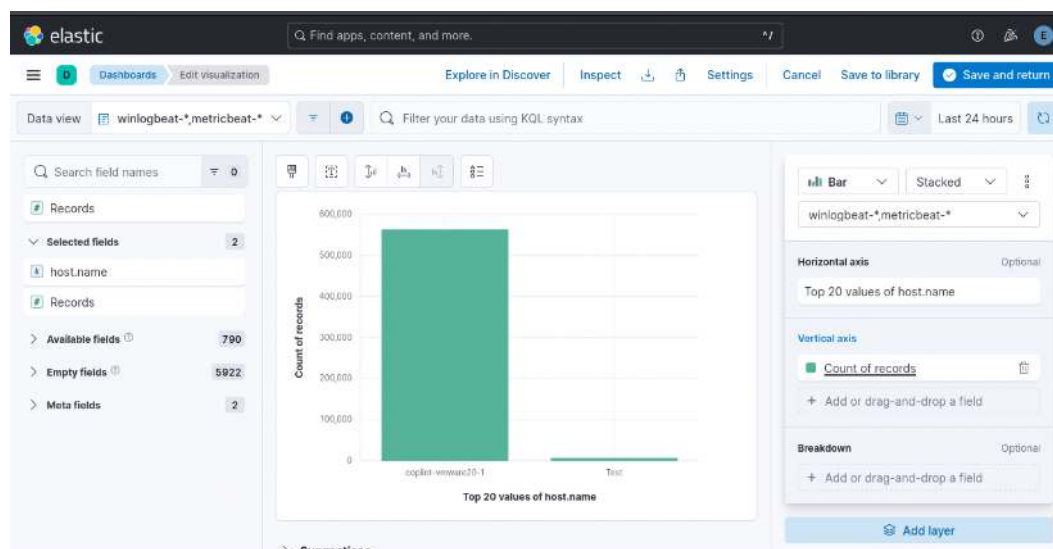


Figure 75: Visualización de barras: cantidad de eventos por `host.name`. Se observa el contraste entre `copilot-vmware20-1` (Metricbeat, ~560k registros) y `Test` (Winlogbeat, ~7k registros).

Se guarda con *Save and return* para volver al dashboard.

9.3 Visualización 2 – Eventos Windows por canal

Paso a paso en Lens:

1. Tipo de gráfica: **Pie** (o *Donut*).
2. Data view: `winlogbeat-*`.
3. **Slice by**: `winlog.channel` con *Top values*.
4. **Size by**: Records (Count).
5. Título del panel: *Eventos Windows por canal*.

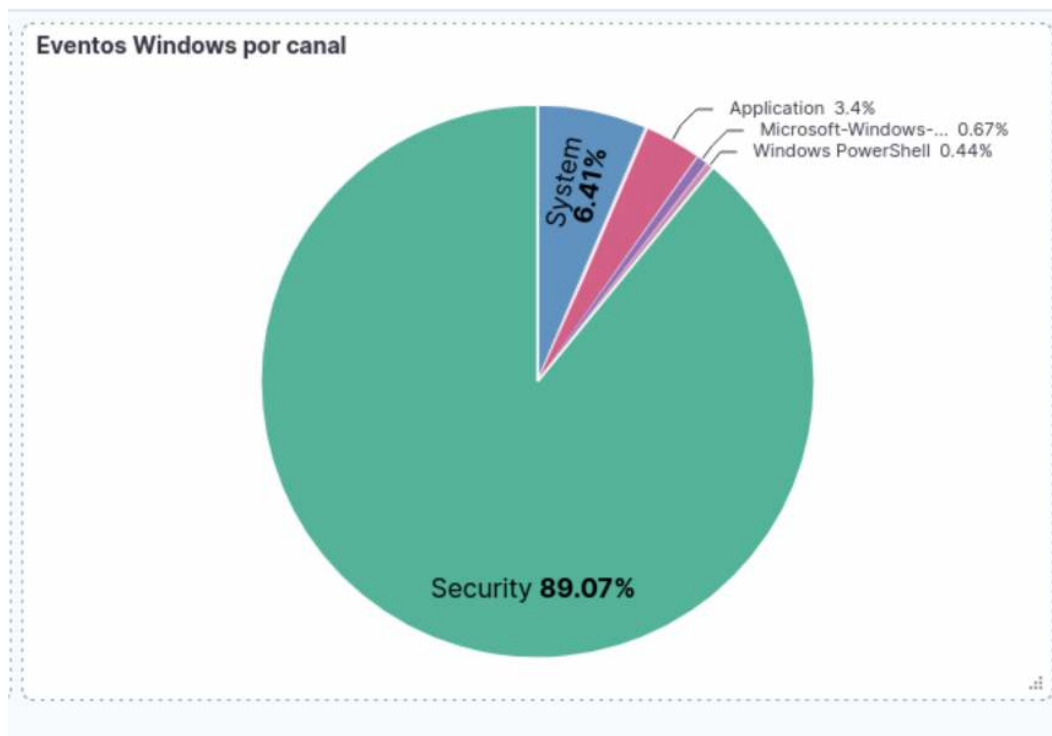


Figure 76: Distribución porcentual de eventos de Winlogbeat por canal: *Security* domina con ~89%, seguido de *System* (6.4%), *Application* (3.4%) y los canales de PowerShell (1.1%).

9.4 Visualización 3 – Eventos por agente

Paso a paso en Lens:

1. Tipo de gráfica: **Bar vertical**.
2. Data view: winlogbeat-*,metricbeat-*.
3. **Horizontal axis**: agent.type (*Top values*).
4. **Vertical axis**: Records (Count).
5. Título del panel: *Eventos por agente*.

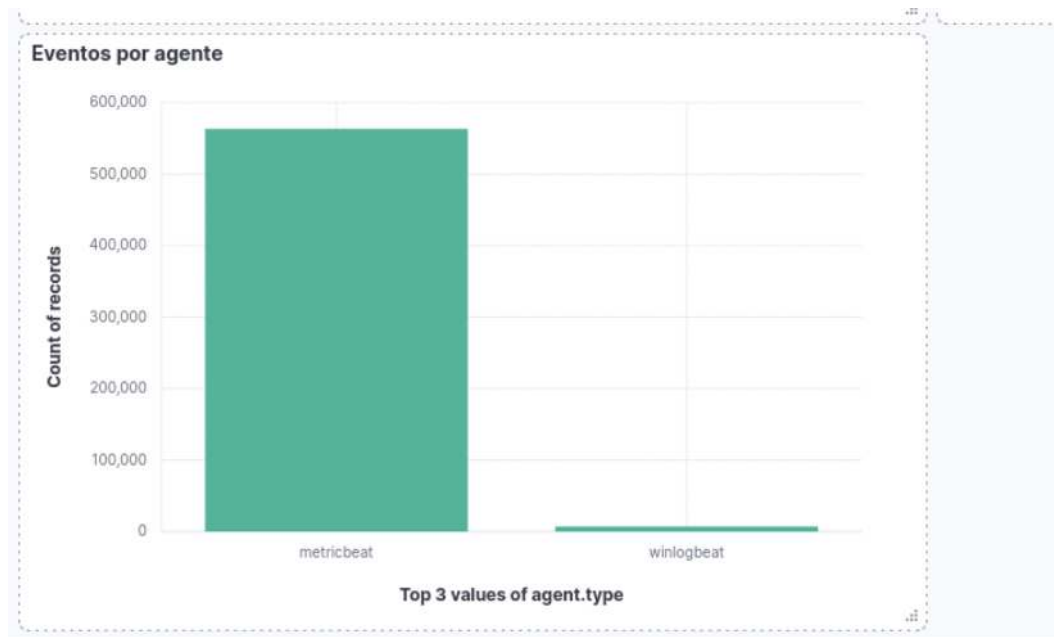


Figure 77: Cantidad de documentos enviados por cada tipo de agente: **metricbeat** aporta el mayor volumen (métricas continuas cada 10s) frente a **winlogbeat** (eventos discretos).

9.5 Visualización 4 – Métricas de CPU de la máquina Linux

Paso a paso en Lens:

1. Tipo de gráfica: **Line**.
2. Data view: **metricbeat-***.
3. Filtro KQL: `event.dataset : "system.cpu"`.
4. **Horizontal axis**: `@timestamp` (Date histogram).
5. **Vertical axis**: `system.cpu.total.norm.pct` con función **Average**.
6. Título del panel: *CPU – Ubuntu (% promedio)*.

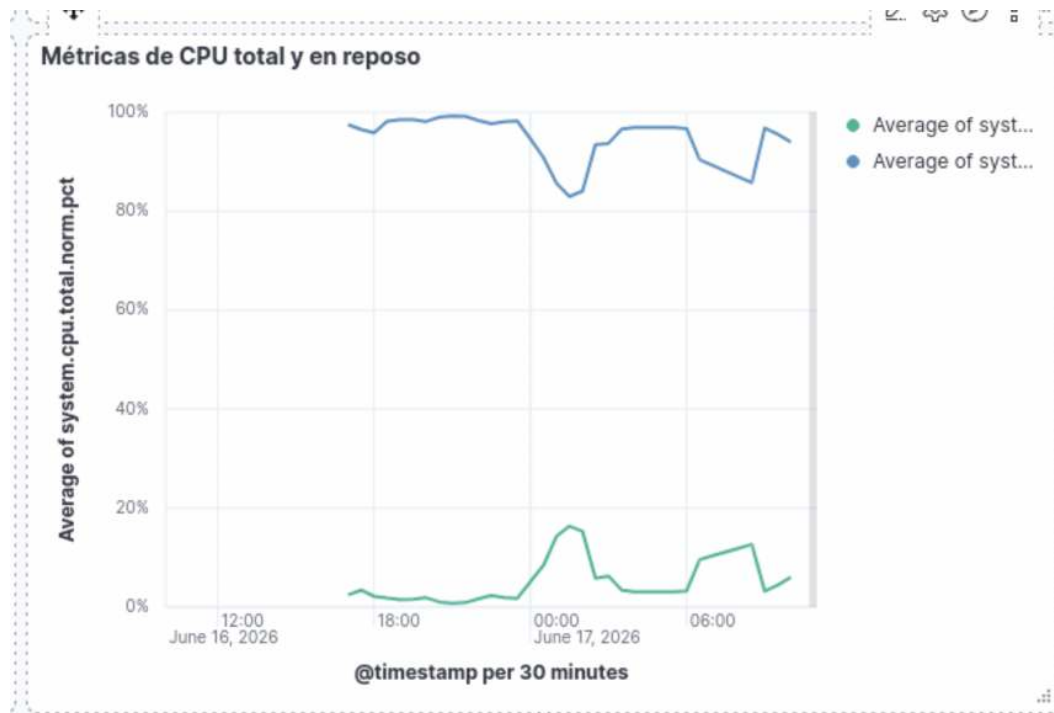


Figure 78: Uso promedio de CPU en la máquina Ubuntu a lo largo del tiempo. Los picos cercanos al 100% corresponden a la prueba de `stress -cpu` de la Fase 6.

9.6 Visualización 5 – Métricas de memoria de la máquina Linux

Paso a paso en Lens:

1. Tipo de gráfica: **Line**.
2. Data view: `metricbeat-*`.
3. Filtro KQL: `event.dataset : "system.memory"`.
4. **Horizontal axis**: `@timestamp`.
5. **Vertical axis**: `system.memory.actual.used.pct` con función **Average**.
6. Título del panel: *Memoria – Ubuntu (% real usado)*.

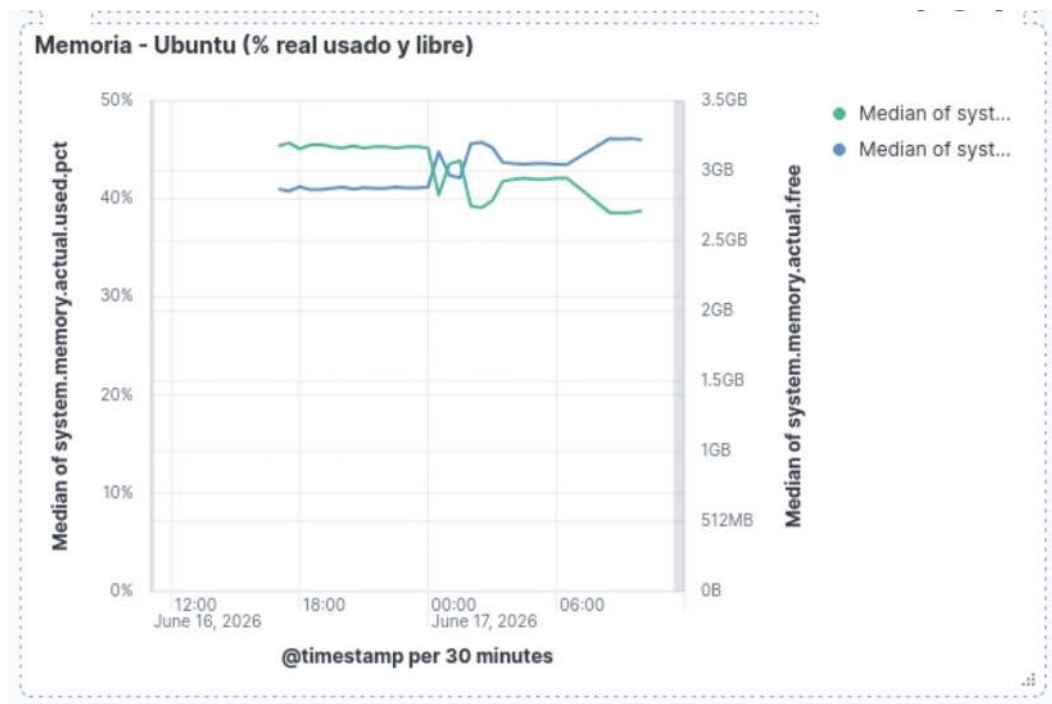


Figure 79: Uso real de memoria en Ubuntu a lo largo del tiempo. El pico hacia el 94% corresponde a la prueba de `stress -vm 1 -vm-bytes 4000M` de la Fase 6.

9.7 Visualización 6 – Uso de disco de la máquina Linux

Paso a paso en Lens:

1. Tipo de gráfica: **Bar vertical** (o **Metric** si se prefiere un solo valor).
2. Data view: `metricbeat-*`.
3. Filtro KQL: `event.dataset : "system.filesystem"`.
4. **Horizontal axis**: `system.filesystem.mount_point`.
5. **Vertical axis**: `system.filesystem.used.pct` con función **Maximum**.
6. Título del panel: *Uso de disco – Ubuntu (% por partición)*.

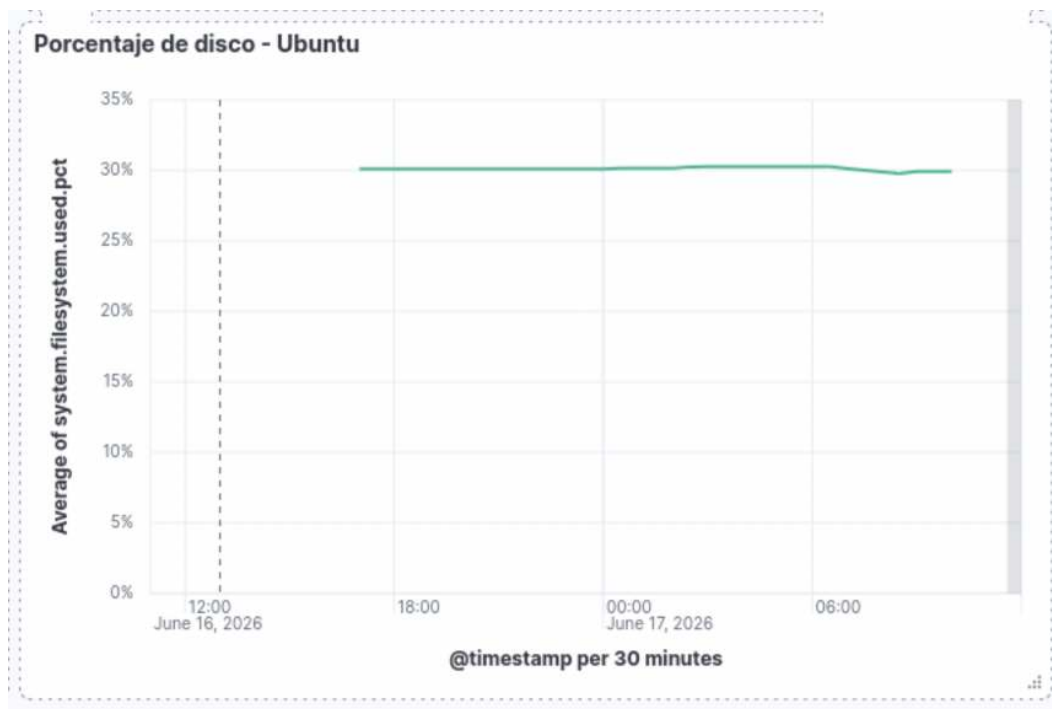


Figure 80: Porcentaje máximo de uso de disco por punto de montaje en Ubuntu.

9.8 Visualización 7 – Línea de tiempo de eventos recibidos

Paso a paso en Lens:

1. Tipo de gráfica: **Area** (*stacked*) o **Line**.
2. Data view: `winlogbeat-*,metricbeat-*`.
3. **Horizontal axis**: `@timestamp` (Date histogram, intervalo automático).
4. **Vertical axis**: **Records** (Count of records).
5. **Breakdown**: `agent.type` (separa las series por agente).
6. Título del panel: *Línea de tiempo de eventos recibidos*.



Figure 81: Línea de tiempo del volumen de eventos recibidos en Elasticsearch, segmentado por `agent.type`. Permite observar la cadencia continua de Metricbeat frente a los picos esporádicos de Winlogbeat.

10 Fase 8: Análisis de seguridad

A continuación se analizan los eventos y métricas recolectados durante el workshop y se responden las preguntas planteadas en la guía.

1. ¿Qué eventos podrían indicar intentos de acceso no autorizado?

Los eventos recolectados que evidencian intentos de acceso no autorizado son principalmente del canal **Security**:

- **4625 (Logon failure)**: capturado durante el workshop con **LogonType: 2** (interactivo) y **FailureReason: Unknown user name or bad password**. Una secuencia de varios 4625 del mismo usuario o IP en pocos segundos es la huella clásica de un ataque de fuerza bruta.
- **4624 (Logon success) con LogonType anómalo**: un **LogonType: 10** (RDP) o **LogonType: 3** (red) desde una IP desconocida fuera del horario laboral es señal de acceso no autorizado.
- **4720 (User account created)** o **4732/4733** (cambios en grupos administrativos) sin un cambio aprobado correspondiente.
- **4724 (Password reset by admin)** no programado.
- **4946/4948 (Firewall rule added/deleted)** sin ticket asociado.
- **4104 (PowerShell Script Block)** con contenido ofuscado, codificado en Base64 o que invoca **Invoke-Expression**, **IEX**, **DownloadString**, etc.

2. ¿Qué eventos podrían indicar fallas operativas?

- **7036 / 7040 / 7045** en el canal **System** cuando indican que servicios críticos cambiaron de estado de forma inesperada.
- **6006 (Event Log service stopped)**: especialmente sospechoso si no fue planeado, ya que detiene el registro de auditoría.
- **1000 (Application Error)** en el canal **Application**: indica crash de aplicaciones.
- **8198 / 8200 (License Activation failure)**: errores operativos de licenciamiento.
- En Linux: **system.service.state = failed** en **event.dataset: system.service**, o servicios cuyo **sub_state** cambia repetidamente entre **auto-restart** y **dead**.

3. ¿Qué métricas podrían ayudar a detectar degradación del sistema?

Las métricas de Metricbeat recolectadas en Ubuntu son altamente útiles para detectar degradación temprana:

- **system.cpu.total.norm.pct > 0.9 sostenido**: saturación de CPU. En el workshop se observó 1.0 (100%) durante la prueba con **stress -cpu**.
- **system.memory.actual.used.pct > 0.9 combinado con system.memory.swap.used.pct > 0**: presión de memoria con inicio de paginación a disco. Se observó 0.9367 y 0.0512 respectivamente durante la prueba con **stress -vm**.
- **system.filesystem.used.pct > 0.85**: riesgo de agotamiento de espacio en disco.
- **system.diskio.read/write.bytes** anómalamente alto: saturación de I/O sin necesariamente llenar el disco (típico de ransomware en plena ejecución).

- `host.network.ingress.packets` con picos abruptos: posible DDoS o exfiltración. Se observaron 101,125 paquetes/intervalo durante la prueba de `ping -f`, frente a 559 en condiciones normales.
- `system.process_summary.total` cambiando bruscamente: aparición masiva de procesos puede indicar fork bomb, minería de criptomonedas o malware.

4. ¿Qué campos son más útiles para investigar un evento?

A partir de los JSON capturados, los campos más útiles para investigación son:

Campo	Uso en investigación
<code>@timestamp</code>	Línea de tiempo: cuándo ocurrió el evento.
<code>host.name, agent.name</code>	Dónde ocurrió: identifica máquina y agente.
<code>event.code</code> / <code>winlog.event_id</code>	Tipo exacto de evento (4624, 4948, etc.).
<code>event.action</code>	Acción semántica (Logon, Process Creation, etc.).
<code>event.outcome</code>	Éxito o fallo de la acción.
<code>winlog.event_data.TargetUserName</code>	Usuario sobre el que se actuó.
<code>winlog.event_data.SubjectUserName</code>	Usuario que ejecutó la acción.
<code>winlog.event_data.IpAddress</code>	IP origen (útil como <code>source.ip</code>).
<code>winlog.event_data.LogonType</code>	Tipo de logon (interactivo, red, servicio, RDP).
<code>winlog.event_data.ProcessName</code>	Binario asociado al evento.
<code>winlog.event_data.ScriptBlockContent</code>	Código literal de scripts PowerShell ejecutados (4104).
<code>event.dataset</code>	Discrimina entre tipos de métrica (<code>system.cpu</code> , <code>system.memory</code> , etc.).

5. ¿Qué información adicional haría falta para mejorar la detección?

- **Logs de aplicaciones** mediante **Filebeat** (Apache, Nginx, syslog, auth.log, bases de datos).
- **Telemetría de red profunda** con **Packetbeat** o **Zeek** para inspeccionar protocolos.
- **Auditoría a nivel kernel en Linux** con **Auditbeat** (cambios en archivos críticos, ejecuciones, etc.).
- **Sysmon** en Windows para enriquecer la visibilidad sobre procesos, conexiones de red, hashes y carga de DLLs.
- **Threat Intelligence Feeds** (IOCs, dominios maliciosos, IPs comprometidas) para correlación.
- **Reglas de detección** (motor de alertas de Kibana Security o equivalente) para automatizar la generación de incidentes.
- **Logs de DNS** para detectar exfiltración o C2.
- **Eventos de Endpoint Detection & Response (EDR)** para correlacionar con la actividad de procesos.

6. ¿Por qué es importante centralizar logs en una organización?

- **Visibilidad única:** el equipo de seguridad puede consultar todos los hosts desde un solo lugar.
- **Correlación cross-system:** un evento en Windows (4625) puede correlacionarse temporalmente con un pico de red en Linux para detectar un ataque coordinado.

- **Preservación de evidencia:** si un atacante limpia los logs locales (`wevtutil cl Security` en Windows), el SIEM aún conserva la copia. Esto incluso permite detectar el propio acto de limpieza (evento 1102).
- **Cumplimiento normativo:** regulaciones como PCI-DSS, ISO 27001 o leyes de protección de datos exigen retención centralizada.
- **Detección temprana:** análisis en tiempo casi real frente a revisar host por host.
- **Auditoría administrativa:** trazabilidad de cambios realizados por administradores en cualquier sistema.

7. ¿Qué ventajas ofrece una plataforma centralizada para un equipo SOC?

- **Único punto de búsqueda:** con KQL o ES|QL un analista puede consultar todos los hosts y todos los tipos de evento.
- **Dashboards reutilizables:** visualizaciones de `system.cpu`, `winlog.event_id`, etc., compartidas por todo el equipo.
- **Filtrado por `agent.type` y `host.name`:** permite escalar a cientos de máquinas sin cambiar la arquitectura; cada nuevo agente solo añade *filas* al mismo índice.
- **Alertas automáticas:** triggers cuando se cumplen condiciones (ej. más de 5 4625 en un minuto desde la misma IP).
- **Roles y permisos centralizados:** distintos perfiles de analista, supervisor y administrador con acceso diferenciado.
- **Triage colaborativo:** varios analistas pueden trabajar sobre el mismo incidente con marcadores y vistas guardadas.
- **Auditoría de la propia herramienta:** el SIEM registra quién consultó qué, importante para el cumplimiento interno.

8. ¿Cómo puede un SIEM apoyar una investigación forense inicial?

- **Línea de tiempo unificada:** el SIEM permite reconstruir cronológicamente todo lo ocurrido en el host afectado y en hosts relacionados.
- **Preservación de evidencia:** los logs ya se encuentran en Elasticsearch antes de que el atacante pueda borrarlos. Por ejemplo, aunque se ejecute `wevtutil cl Security` en Windows, el evento 1102 (*The audit log was cleared*) queda registrado en Elasticsearch.
- **Ciclo de vida de procesos:** con la auditoría de *Process Creation* (4688) y *Process Termination* (4689) se puede reconstruir qué binarios se ejecutaron, con qué línea de comandos y por cuánto tiempo.
- **Reconstrucción de actividad de usuarios:** combinando 4624 (login), 4634/4647 (logoff), 4720 (account created) y 4732/4733 (group changes) se construye la actividad administrativa completa.
- **Correlación entre fuentes:** un pico en `system.cpu.total.pct` en Linux que coincide en el tiempo con un 4104 (PowerShell) en Windows sugiere actividad coordinada posiblemente maliciosa.
- **Búsqueda histórica:** consultas sobre el histórico (no solo en tiempo real) permiten reconstruir incidentes detectados tardíamente.

9. ¿La implementación realizada puede considerarse un SIEM completo? Justifique la respuesta.

No, no puede considerarse un SIEM completo. La implementación cubre los componentes *básicos* de un SIEM (recolección, indexación, búsqueda y visualización), pero carece de varios elementos esenciales para un entorno productivo:

Lo que SÍ tiene la implementación

- Recolección de logs (Winlogbeat).
- Recolección de métricas (Metricbeat).
- Almacenamiento e indexación centralizada (Elasticsearch).
- Búsqueda y visualización (Kibana/Discover/Dev Tools).
- Capacidad de correlación manual mediante consultas combinadas en ES|QL.

Lo que le falta para ser un SIEM completo

- **Motor de reglas y alertas** configurado (Kibana Security Detection Rules, Watcher, o equivalente).
- **Threat Intelligence** integrado.
- **Respuesta orquestada (SOAR)** para acciones automáticas ante incidentes.
- **Logs de aplicaciones** (Filebeat) y **telemetría de red** (Packetbeat) están ausentes.
- **Solo se monitorean dos hosts**, lo que no representa una cobertura organizacional real.
- **Seguridad de la plataforma**: se usa HTTP en Kibana en vez de HTTPS, las credenciales viajan en texto plano dentro de los archivos `.yaml` de los Beats, y no se implementa RBAC granular.
- **Retención**: no se definió política ILM personalizada; los índices podrían crecer indefinidamente.
- **Casos de uso documentados**: un SIEM completo requiere *playbooks* de respuesta para los principales tipos de incidente.

En consecuencia, la implementación realizada es un **SIEM funcional de nivel laboratorio o educativo**, válida para demostrar los conceptos y operar con un volumen reducido de hosts, pero no apta para producción sin endurecimiento adicional.

10. ¿Qué problemas podrían surgir si se recolectan demasiados eventos sin una estrategia clara?

- **Costos de almacenamiento**: índices grandes consumen disco rápido y costoso (SSD). En el workshop, `metricbeat-*` ya genera un orden de magnitud más documentos que `winlogbeat-*` (ver Fase 5).
- **Degradación de rendimiento**: consultas lentas en Elasticsearch cuando los índices superan ciertos tamaños sin optimización (ILM, shards adecuados, hot/warm/cold).
- **“Alert fatigue”**: un SOC inundado de alertas irrelevantes ignora también las importantes. Por ejemplo, el evento 4624 con `LogonType: 5` (servicios) genera cientos de documentos por hora sin valor de seguridad.

- **Pérdida de señal entre ruido:** si todos los canales se recolectan al máximo nivel, los eventos críticos quedan enterrados entre eventos triviales (ej. `Microsoft-Windows-Security-SPP` con eventos de activación de Windows).
- **Compliance y privacidad:** recolectar logs sin estrategia puede incluir datos personales (PII) que disparan obligaciones regulatorias (GDPR, HIPAA, ley colombiana 1581 de 2012).
- **Costo de procesamiento:** los *pipelines* de ingesta de Elasticsearch consumen CPU; demasiados pueden saturar el nodo.
- **Dificultad de mantenimiento:** añadir más datasets sin un *schema* consistente y nomenclatura clara complica el análisis y la creación de dashboards.

Conclusión: la estrategia correcta es definir *casos de uso de seguridad* primero y recolectar únicamente los datos necesarios para soportarlos, con filtrado, normalización y políticas ILM adecuadas. Calidad por encima de cantidad.